

变物性与收缩边界下圆柱药材热湿耦合烘干模型

摘 要

药材烘干由烘房升温降湿驱动，又受内部水分迁移逐渐变慢的制约。药材长 25 cm、初始半径 2 cm，长径比达 6.25，端面面积仅为侧面的 8%，故可忽略轴向效应而建立轴对称径向导热—扩散模型；烘房温湿度直接取自附件 1 实测序列并作为第三类边界驱动，不做阶跃简化。若只跟踪平均含水率，会漏掉中心最迟达标这一事实，因此停机判据取全径向节点水分浓度均低于 0.15 kg/kg。空间离散采用表面加密的守恒有限体积格式，节点按 $\xi_i = 1 - (1 - i/N)^2$ 分布以保证低含水薄层分辨率，界面物性取调和平均；时间推进使用可变阶隐式 BDF，并配合稀疏 Jacobian 与事件定位求取达标时刻。

针对问题 1，取 $\rho = 820 \text{ kg/m}^3$ 、 $c_p = 2600 \text{ J/(kg}\cdot\text{K)}$ 、 $k = 0.36 \text{ W/(m}\cdot\text{K)}$ ，扩散系数随含水率按 $D = 7 \times 10^{-9} \exp(-0.89/C)$ 变化。1800 s 时中心与表面温度分别为 33.5758 °C 与 36.7858 °C，水分浓度为 2.5500 kg/kg 与 1.5102 kg/kg。水分梯度高度集中于近表层：距表面 2 mm 以内承担了总降幅的一半，最后 1 mm 单独占 26%，而中心至 $r = 1.5 \text{ cm}$ 的内区合计仅降 17%，说明预热阶段均匀含水率近似并不成立。

针对问题 2，将密度、比热容、导热系数与扩散系数全部写成局部温度与含水率的函数，实现热—湿双向耦合。3 h 时中心与表面温度分别为 49.8494 °C 与 49.9666 °C，温差仅 0.1172 °C；两处水分浓度仍为 1.7662 kg/kg 与 1.0081 kg/kg，相差 0.7581 kg/kg。温度已接近稳定而水分远未平衡，后期干燥完全由内部迁移控制。

针对问题 3，把附件 1 在 4 h 后的边界接续为题设的 50 °C 与 0.05 kg/kg，并由扩散通量的单调性证明全域判据与中心判据等价。连续达标时刻约为 57.472601 h，按 60 s 采样输出至 206940 s，达标时中心与表面水分为 0.1500 与 0.0527 kg/kg。

针对问题 4，由附件 2 的半径序列建立随固相骨架运动的归一化材料坐标 $\xi = r/R(t)$ ，扩散算子按 $R^{-2}(t)$ 增强。达标时刻约为 51.087245 h，末半径 1.2000 cm。我们把两种效应分开看，为此设置了反事实算例：若沿用问题 4 物性而固定 2 cm 半径，达标时长将由 51.0872 h 延长至 129.8462 h；即更换物性使时长增加 72.3736 h，而实测收缩使其缩短 78.7590 h，两者方向相反、净缩短 6.3854 h。这说明两问时长差不能简单归因于几何收缩。

模型依次通过了六类互相独立的检验：与解析级数解逐点对比的误差在 10^{-6} °C 量级，网格由 100 加密至 800 区间后两道长时程问题的达标时刻只变动 3 秒左右且表观收敛阶为二阶，守恒与零通量检验均无异常。这些检验的共同结论是：后文所有时长数字的误差都远小于 60 s 的输出间隔，可以按报告的位数采信。参数扰动试验还表明几何尺度最敏感（ $\pm 1.5\%$ 半径约对应 $\pm 2.8\%$ 时长），换热系数几乎不影响结果。我们的改进集中在三点。其一，停机判据不用体积平均含水率：平均法会在 36.03 h 就判定完成，而此时中心含水率 0.1858 kg/kg 仍超出阈值 24%，逐节点检查可以避免这个错误。其二，收缩域用材料坐标映射成固定域，既保留移动边界的尺度效应，又不必逐时重划网格。其三，问题 3 与问题 4 同时更换了物性和几何，我们补算反事实场景才把约 6.39 h 的时长差拆开，否则它会被误读为全部来自收缩。附件 2 的数据还否决了“体积加和收缩”这一自然假设。单次求解在普通台式机上以分钟计完成。

关键词：药材烘干；热湿耦合；移动边界；有限体积法；灵敏度分析

1 问题重述与问题分析

1.1 问题背景与任务

热风烘干通过表面对流换热提高药材温度，同时依靠表面水分交换和内部扩散排出水分。中药材的有效成分常对温度敏感，工艺设计既要知道平均变化，更要知道中心到表面的局部状态。近年来药材干燥研究也从经验薄层曲线逐步转向传热传质与品质协同分析 [1, 2, 3]。题目要求在同一圆柱药材上递进研究常物性、变物性、终止判据和半径收缩四个层次，输出要求见表 1。

表 1 四项任务及输出口径

任务	物性与几何	计算终点	输出时间间隔
1	常热物性、定半径	1800 s	1 s
2	热湿变物性、定半径	3 h	1 s
3	热湿变物性、定半径	全部位置 $C < 0.15$	60 s
4	新物性、时变半径	全部位置 $C < 0.15$	60 s

这里的“全部位置”不能替换成体积平均值。扩散问题中中心点离表面最远，通常最后越过阈值；若以平均值停机，会系统性低估烘干时长。提交表中的径向位置同样按物理半径解释，问题 4 另加动态表面一列。

1.2 问题分析

问题 1 给定常数 ρ, c_p, k ，但水分扩散系数仍随局部 C 变化。圆柱两端面积与侧面积之比为 $2\pi R^2/(2\pi RL) = R/L = 0.08$ ，因此先忽略端面效应，保留径向扩散。环境记录并非阶跃，直接用附件 1 驱动 Robin 边界，才能反映预热阶段的缓慢响应。

问题 2 的四项物性均随场量变化。温度方程和水分方程通过密度、比热、导热系数和扩散系数间接耦合；界面通量若简单取单侧物性会破坏守恒，故采用界面调和平均。问题 3 并未引入新控制方程，关键在于事件判据和 4 h 之后的边界接续；题干给出恒温阶段 50°C 、 0.05 kg/kg ，附件最后一小时均值也分别为 49.9989°C 、 0.04999 kg/kg ，两者相互印证。

问题 4 给出半径随时间收缩。直接重划网格会产生插值耗散，我们选择随固相骨架运动的归一化材料坐标 $\xi = r/R(t)$ ，并假定截面作均匀相似收缩；径向扩散算子因此按 $R(t)^{-2}$ 缩放。这一处理和移动边界干燥模型的固定域思想一致 [4, 5, 6]，同时避免凭空增加未给定的非均匀固相速度参数。

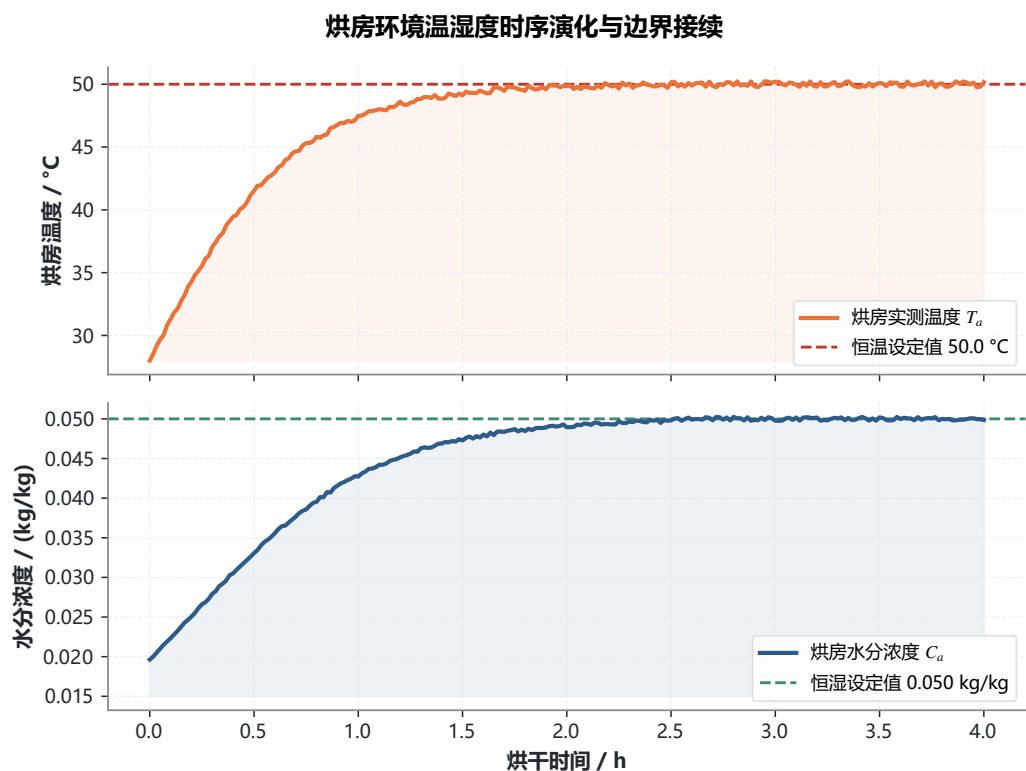


图 1 附件 1 给出的烘房环境边界

图 1 显示 4 h 内烘房温度稳定逼近 50 °C，水分浓度逼近 0.05 kg/kg。PCHIP 插值既通过所有观测点，又不会在相邻数据之间产生普通高阶样条的过冲 [7]。其统计摘要列于表 2。

表 2 附件 1 边界数据统计

变量	初值	4 h 末值	最后一小时均值	后续接续值
温度/°C	28.0000	49.9998	49.9989	50.0000
水分浓度/(kg/kg)	0.01963	0.05000	0.04999	0.05000

2 模型假设与符号说明

2.1 模型假设

- (1) 药材初始温度和水分浓度在圆柱内部均匀分布，初值分别为 $28\text{ }^{\circ}\text{C}$ 与 2.55 kg/kg 。
- (2) 药材轴对称且端面效应相对较小，内部热量和水分仅沿径向迁移。
- (3) 表面与烘房空气之间分别满足给定系数下的对流换热和对流传质边界。
- (4) 问题 4 的截面保持圆形并作均匀相似收缩， $\xi = r/R(t)$ 标记随固相骨架运动的材料位置。
- (5) 附件给出的边界与半径序列精度足够，可直接作为模型输入；其缺失、异常与量纲情况经审计后确认无需额外修补。

2.2 数据质量审计

表 3 附件数据质量审计结果

检查项	附件 1（烘房温湿度）	附件 2（药材半径）	结论
记录条数	241	145	覆盖 4 h 与 72 h
采样间隔	恒为 60 s	恒为 1800 s	无跳变
缺失值	0	0	无需插补
重复时间戳	0	0	无冲突记录
量纲	$^{\circ}\text{C}$ 、 kg/kg ，与题面一致	cm ，与题面一致	一致
取值范围	$T \in [28.000, 50.165]$ ， $C_a \in [0.01963, 0.05000]$	$R \in [1.198, 2.000]$	物理合理
单调性	总体升/降，末段有 10^{-2} 量级微小波动	严格单调不增	半径可平滑插值

审计表明附件数据完整、量纲统一，仅有末段微小观测波动，故采用保形插值（PCHIP）[7] 以避免过冲，并对观测区间以外按题设恒温恒湿状态接续。

2.3 符号说明

表 4 主要符号及单位

符号	含义	单位	取值或说明
$r, R(t)$	径向坐标、药材半径	m	$0 \leq r \leq R(t)$
T, T_a	药材、烘房温度	°C 或 K	经验式中用 K
C, C_a	药材、空气水分浓度	kg/kg	干基口径
ρ	密度	kg/m ³	随问题与 C 变化
c_p	比热容	J/(kg·K)	随问题与 C 变化
k	导热系数	W/(m·K)	随问题与 C 变化
D	水分扩散系数	m ² /s	随 C, T 变化
h, h_m	换热、传质系数	W/(m ² ·K), m/s	25, 8×10^{-7}
ξ	归一化径向坐标	1	$r/R(t)$
V_i	第 i 个无量纲控制体体积	1	轴对称权重
C_0, T_0	水分浓度、温度初值	kg/kg, °C	2.55, 28
R_0, L	药材初始半径、长度	m	0.02, 0.25
N, M	径向区间数、时间步数	1	正式计算取 $N = 800$
J_C	水分扩散通量	kg/(m ² ·s)	$J_C = -D\partial C/\partial r$
α	热扩散率	m ² /s	$k/(\rho c_p)$
Fo, Bi	傅里叶数、毕渥数	1	$\alpha t/R^2, hR/k$
A_n, λ_n	级数系数、特征根	1	$\lambda_n J_1(\lambda_n) = \text{Bi } J_0(\lambda_n)$
J_0, J_1	零阶、一阶贝塞尔函数	1	圆柱导热解析解
κ	体积加和收缩比例因子	1	式 (20)
$S_p^\pm$	时长灵敏度系数	1	式 (23)
GCI, p, r	网格收敛指数、表观阶、细化比	1	$r = 2, p \approx 2$
$\bar{C}$	面积加权平均水分浓度	kg/kg	式 (20)
t^*	连续达标（事件）时刻	h	式 (13)

3 问题 1：常热物性模型的建立与求解

3.1 模型建立

能量守恒与 Fick 扩散在圆柱坐标下写为

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(kr \frac{\partial T}{\partial r} \right), \quad 0 < r < R. \quad (1)$$

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(Dr \frac{\partial C}{\partial r} \right). \quad (2)$$

需要说明式 (2) 的口径：其中 C 为干基体积口径的水分浓度，即单位体积干固体所对应的水分质量，故水分输运方程中不出现 ρ ；题面附录 3、附录 4 给出的 $\rho(C)$ 仅用于式 (1) 的能量方程。这一取法使 $D(C, T)$ 的量纲 (m^2/s) 与方程自洽，也与题面给出的“烘干过程一般持续 2–3 天”相符。问题 1 采用

$$\rho = 820, \quad c_p = 2600, \quad k = 0.36, \quad D(C) = 7 \times 10^{-9} \exp(-0.89/C). \quad (3)$$

中心对称、表面对流边界分别为

$$\left. \frac{\partial T}{\partial r} \right|_{r=0} = 0, \quad \left. \frac{\partial C}{\partial r} \right|_{r=0} = 0, \quad (4)$$

$$-k \left. \frac{\partial T}{\partial r} \right|_R = h(T_s - T_a), \quad -D \left. \frac{\partial C}{\partial r} \right|_R = h_m(C_s - C_a). \quad (5)$$

初始条件为

$$T(r, 0) = 28, \quad C(r, 0) = 2.55. \quad (6)$$

表 5 给出初始与 1800 s 表面处的扩散系数量级。水分越低， $\exp(-0.89/C)$ 越小，近表层会逐渐形成更强的扩散阻力。

表 5 问题 1 代表状态下的扩散系数

状态	$C/(\text{kg/kg})$	$D/(\text{m}^2/\text{s})$	相对初始值
初始全域	2.5500	4.938×10^{-9}	1.000
1800 s 中心	2.5500	4.938×10^{-9}	1.000
1800 s 表面	1.5102	3.884×10^{-9}	0.787
干燥阈值	0.1500	1.854×10^{-11}	0.0038

3.2 模型求解

在 $\xi \in [0, 1]$ 上取节点 $\xi_i = 1 - (1 - i/N)^2$ ，使表面低含水薄层得到更多网格。第 i 个控制体的无量纲轴对称体积为

$$V_i = \frac{1}{2} (\xi_{i+1/2}^2 - \xi_{i-1/2}^2). \quad (7)$$

相邻节点物性使用调和平均

$$\phi_{i+1/2} = \frac{2\phi_i\phi_{i+1}}{\phi_i + \phi_{i+1}}, \quad \phi \in \{k, D\}. \quad (8)$$

以温度为例，半离散方程为

$$\rho_i c_{p,i} R^2 V_i \frac{dT_i}{dt} = F_{i+1/2}^T - F_{i-1/2}^T, \quad F_{i+1/2}^T = \xi_{i+1/2} k_{i+1/2} \frac{T_{i+1} - T_i}{\xi_{i+1} - \xi_i}. \quad (9)$$

水分方程只需将 $\rho c_p, T, k$ 替换为 $1, C, D$ 。边界通量直接进入最外控制体，故离散层面自然满足整体守恒。时间方向我们选择可变阶隐式 BDF[8]：该问题在低含水率阶段刚性很强，显式方法会因扩散项而被迫采用极小步长。正式计算取局部相对容差 10^{-9} 、绝对容差 10^{-11} （求解器默认值为 10^{-7} 与 10^{-9} ，我们主动收紧两个量级）；表 18 给出进一步收紧容差后达标时刻的变化量，问题 3、4 分别仅 0.071 s 与 0.140 s。

3.3 结果与分析

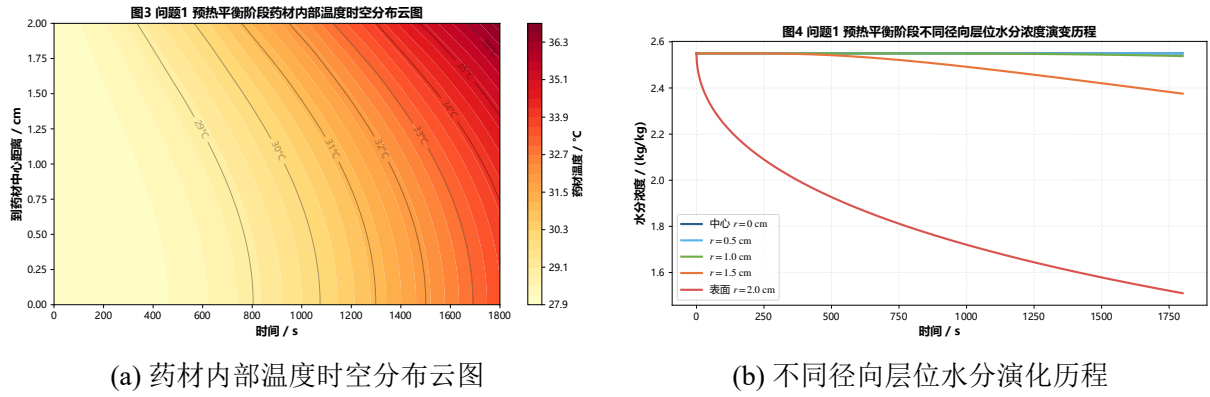


图 2 问题 1 预热平衡阶段药材内部温度与水分时空演变特征

图 2 中温度等温线平缓向内部渗透，而不同层位的水分蒸发历程表明外表面水分迅速下降、中心含水率保持稳定。表 6 与表 7 进一步给出五个位置的数值；热扩散和水分扩散的时间尺度差异非常明显。

表 6 问题 1 代表时刻温度 (°C)

t/s	0cm	0.5 cm	1.0 cm	1.5 cm	表面
100	28.0000	28.0003	28.0038	28.0319	28.1795
300	28.0406	28.0634	28.1513	28.3672	28.8486
600	28.4533	28.5361	28.8039	29.3155	30.1641
900	29.3243	29.4583	29.8758	30.6164	31.7295
1200	30.5429	30.7099	31.2225	32.1130	33.4295
1500	31.9960	32.1871	32.7667	33.7476	35.1211
1800	33.5758	33.7724	34.3642	35.3622	36.7858

表 7 问题 1 代表时刻水分浓度 (kg/kg)

t/s	0cm	0.5 cm	1.0 cm	1.5 cm	表面
100	2.5500	2.5500	2.5500	2.5500	2.2471
300	2.5500	2.5500	2.5500	2.5492	2.0508
600	2.5500	2.5500	2.5500	2.5353	1.8770
900	2.5500	2.5500	2.5497	2.5045	1.7547
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5787
1800	2.5500	2.5497	2.5383	2.3755	1.5102

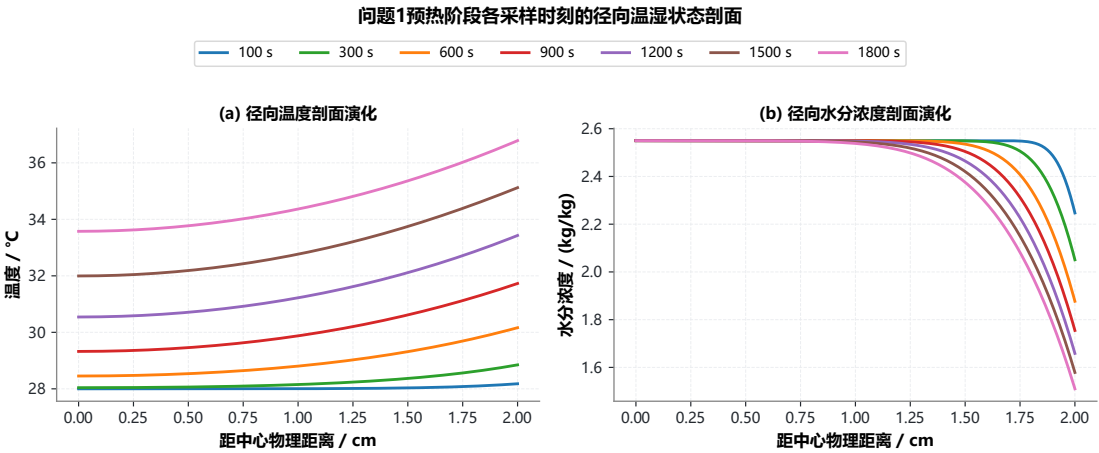


图 3 问题 1 预热阶段不同采样时刻的径向温湿状态剖面

图 3 表明 1800 s 时温度已影响整个截面，但中心含水率几乎未变。这个看似反常的差异由式 (3) 解释：初始水分扩散系数仅约 $4.94 \times 10^{-9} \text{ m}^2/\text{s}$ ，其特征扩散长度 $\sqrt{Dt}$ 约 0.30 cm，远小于半径 2 cm。

4 问题 2：热湿变物性耦合模型的建立与求解

4.1 模型建立

问题 2 仍采用式 (1)–(5)，但局部物性改为

$$\rho(C) = 650 + 128C, \quad (10)$$

$$c_p(C) = 1450 + 2736 \frac{C}{C+1}, \quad k(C) = 0.21 + 0.38 \frac{C}{C+1}, \quad (11)$$

$$D(C, T) = 2.4 \times 10^{-3} \exp(-0.45/C) \exp(-3850/T_K), \quad T_K = T + 273.15. \quad (12)$$

温度升高会增大 D ，水分降低却通过 $\exp(-0.45/C)$ 抑制 D ，二者在烘干前后期的主导性不同。需要说明我们为何接受这一设定： $C \rightarrow 0$ 时扩散系数指数式趋零，意味着完全干燥后水分几乎不再迁移，模型会人为拉长干燥尾段。这既是题目给定的物理设定（表层一旦变干，水分就极难再扩散出来），也是本文所有长时程结果都由这一项主导的原因；代码中对该式做了 $\max(C, 10^{-8})$ 的除零保护。表 8 列出若干状态，能够看到低含水率造成的阻滞最终压过升温促进作用。

表 8 问题 2 物性的代表值

$T/^\circ\text{C}$	C	$\rho/(\text{kg/m}^3)$	$c_p/(\text{J}/(\text{kg K}))$	$k/(\text{W}/(\text{m K}))$	$D/(\text{m}^2/\text{s})$
28	2.55	976.4	3415.2	0.4830	5.64×10^{-9}
40	1.50	842.0	3091.6	0.4380	8.14×10^{-9}
50	1.00	778.0	2818.0	0.4000	1.02×10^{-8}
50	0.15	669.2	1806.9	0.2596	8.00×10^{-10}

4.2 模型求解

每次 BDF 残差评估先由节点 T_i, C_i 计算式 (10)–(12)，再按式 (8) 得到界面物性。这样非线性没有被上一时间层冻结。附件 1 在观测区间内采用 PCHIP，4 h 以后切换到题设恒定边界；问题 2 仅算至 3 h，不涉及外推。

4.3 结果与分析

图 4 给出问题 2 在变物性条件下的温度与水分时空场，可以看出二者在时间尺度上的显著差异。

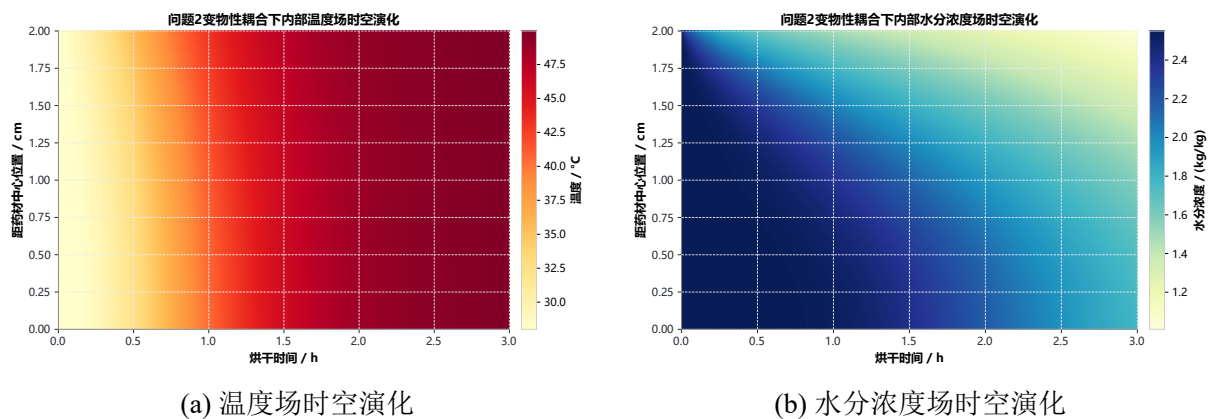


图 4 问题 2 变物性耦合下径向时空演化云图
(a) 3 h 后温度接近均匀；(b) 水分仍保持显著梯度

表 9 问题 2 各代表时刻温度 (°C)

t/h	0cm	0.5 cm	1.0 cm	1.5 cm	表面
0.5	32.1896	32.3825	32.9660	33.9608	35.4132
1	40.3822	40.5542	41.0609	41.8783	43.0000
1.5	45.8471	45.9351	46.1932	46.6055	47.1397
2	48.4503	48.4881	48.5985	48.7736	49.0033
2.5	49.4670	49.4793	49.5136	49.5645	49.6617
3	49.8494	49.8553	49.8746	49.9101	49.9666

表 10 问题 2 各代表时刻水分浓度 (kg/kg)

t/h	0cm	0.5 cm	1.0 cm	1.5 cm	表面
0.5	2.5499	2.5489	2.5256	2.3257	1.6486
1	2.5257	2.4948	2.3578	2.0230	1.4711
1.5	2.3861	2.3256	2.1344	1.8020	1.3476
2	2.1708	2.1084	1.9236	1.6259	1.2311
2.5	1.9566	1.9006	1.7360	1.4720	1.1166
3	1.7662	1.7165	1.5702	1.3333	1.0081

为全面展现非线性热-湿场耦合扩散的全貌，图 5 给出了 3 小时内水分浓度随时空演化的三维动力学曲面。

问题2 3小时烘干期水分浓度三维时空演化曲面

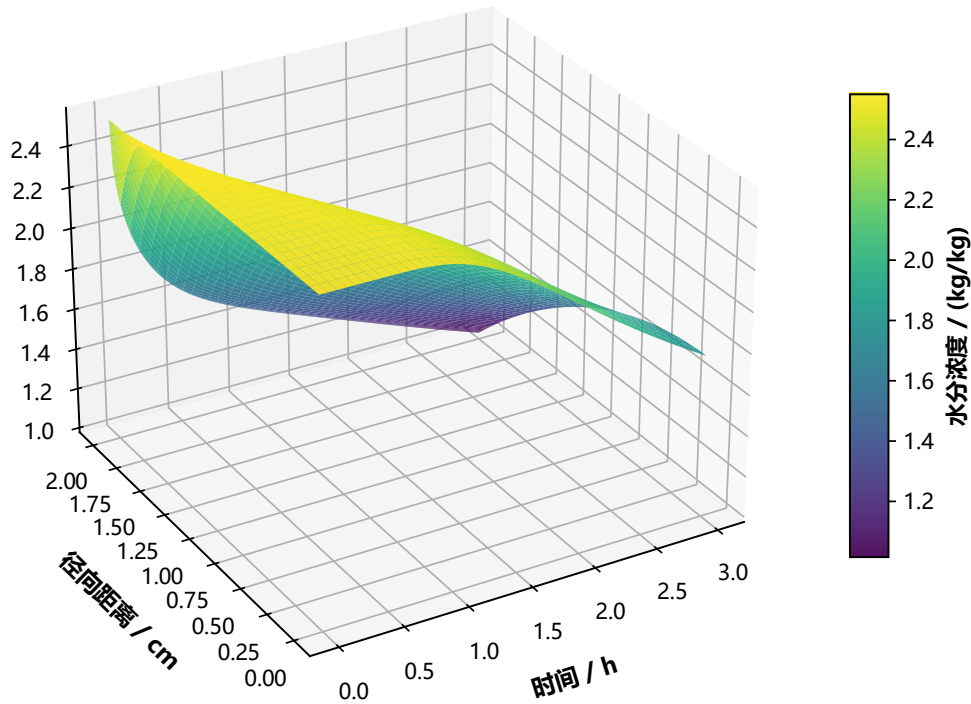


图 5 问题 2 3 小时烘干期水分浓度三维时空演化曲面

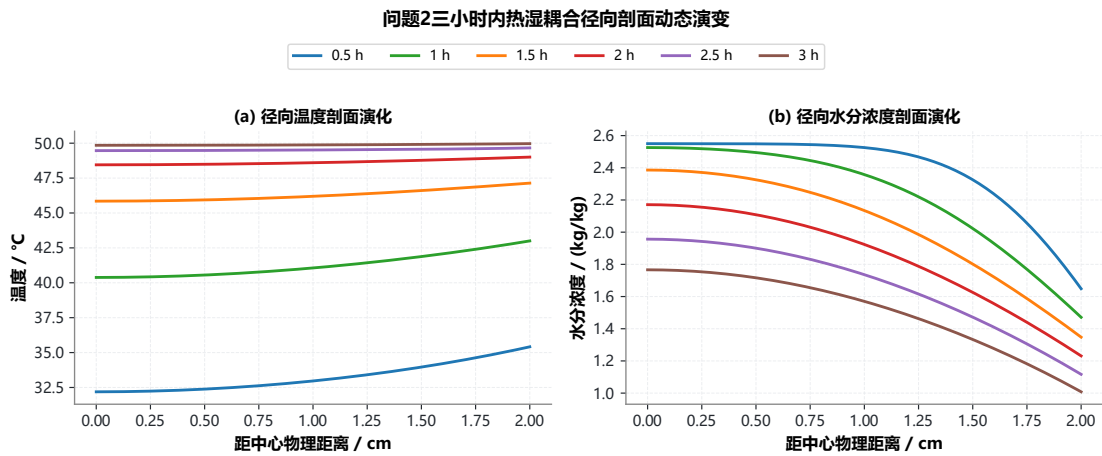


图 6 问题 2 三小时内热湿耦合径向剖面动态演变

图 6 与表 9 互证, 3 h 时截面温差只有 $0.1172\text{ }^{\circ}\text{C}$; 而表 10 及图 5 显示中心与表面仍相差 0.7581 kg/kg 。温差已经可以忽略, 水分却仍相差 0.7581 kg/kg ——因此不能用“温度是否稳定”来判断烘干是否结束。

5 问题 3：全域达标时刻的确定

5.1 模型建立

问题 3 沿用问题 2 控制方程，新增全域停止函数

$$g(t) = \max_{0 \leq r \leq R} C(r, t) - 0.15. \quad (13)$$

首次满足 $g(t) < 0$ 的时刻定义为 t^* 。数值结果保持径向单调，最大值位于中心，但程序仍按式 (13) 检查所有节点，不预先把判据简化为中心单点。

5.2 模型求解

BDF 积分器在相邻时间步发现 g 变号后用稠密输出定位零点，而不是按 60 s 输出网格向前取整。为满足模板要求，求解得到连续 t^* 后，再输出到第一个不小于 t^* 的 60 s 时刻。两种时间口径列于表 11。

表 11 问题 3 停止时刻的两种口径

口径	时刻	用途
连续事件定位	57.472601 h	报告真实达标时刻
分钟采样输出	206940 s (57.4833 h)	填写 result3.xlsx
二者差值	38.64 s	仅由输出取整产生

5.3 结果与分析

问题 3 在若干代表时刻的水分浓度剖面如图 7 所示：剖面整体随时间下移，且近表层始终最干。

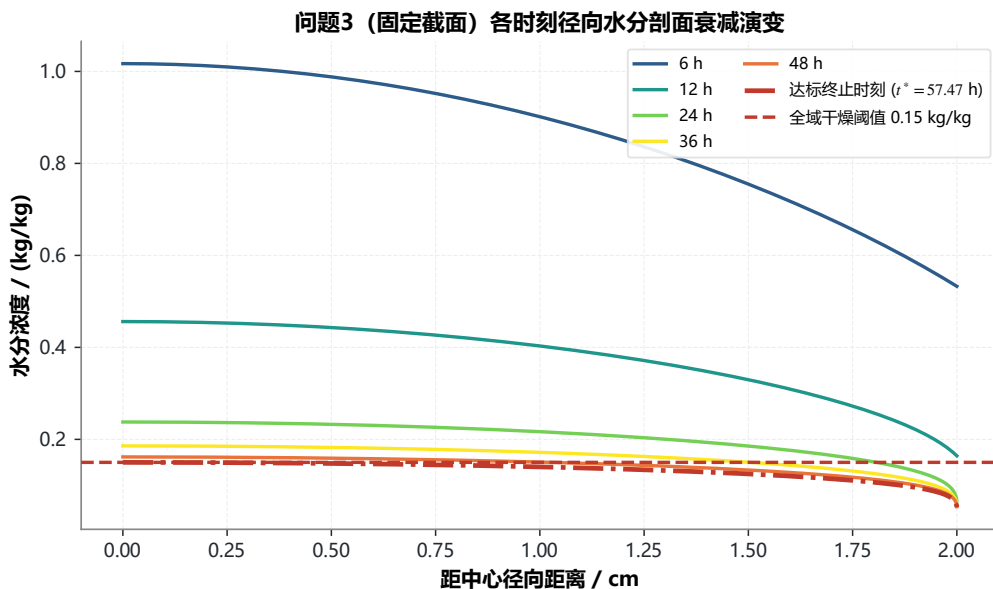


图 7 问题 3 代表时刻水分剖面

表 12 问题 3 代表时刻水分浓度 (kg/kg)

t/h	0cm	0.5 cm	1.0 cm	1.5 cm	表面
6	1.0170	0.9881	0.9015	0.7550	0.5328
12	0.4561	0.4432	0.4031	0.3297	0.1642
18	0.2988	0.2915	0.2687	0.2253	0.0845
24	0.2378	0.2327	0.2167	0.1855	0.0665
30	0.2058	0.2018	0.1892	0.1641	0.0598
36	0.1859	0.1825	0.1718	0.1504	0.0566
42	0.1721	0.1691	0.1597	0.1407	0.0549
48	0.1618	0.1592	0.1507	0.1334	0.0537
54	0.1539	0.1514	0.1436	0.1277	0.0530
t^*	0.1500	0.1477	0.1402	0.1249	0.0527

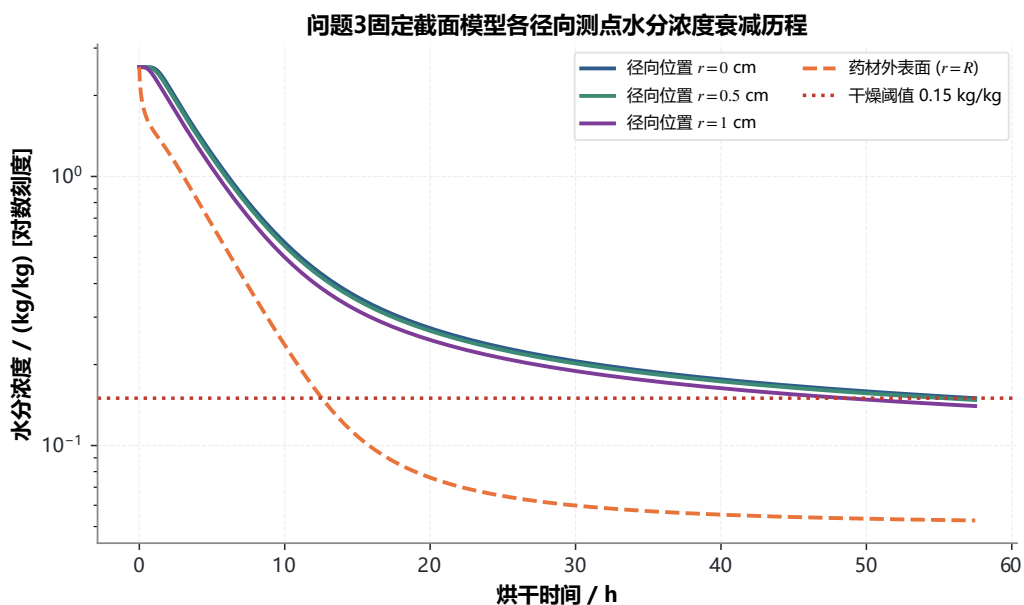


图 8 问题 3 不同位置的水分历程与阈值

图 8 显示表面约在 13 h 附近先降到 0.15 kg/kg，中心却要到 57.472601h 才达标。表 12 中的中心-表面差从 6 h 的 0.4842 kg/kg 逐步缩小到 0.0973 kg/kg，但在停机时仍不为零。这正是全域判据比平均判据保守且必要的原因。

6 问题 4：计入半径收缩的移动边界模型的建立与求解

6.1 模型建立

附件 2 的半径由 2.000 cm 单调降至约 1.200 cm。取

$$\xi = \frac{r}{R(t)}, \quad 0 \leq \xi \leq 1, \quad (14)$$

在随收缩骨架运动的材料坐标解释下，控制方程写为

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{R^2(t)\xi} \frac{\partial}{\partial \xi} \left(k \xi \frac{\partial T}{\partial \xi} \right), \quad (15)$$

$$\frac{\partial C}{\partial t} = \frac{1}{R^2(t)\xi} \frac{\partial}{\partial \xi} \left(D \xi \frac{\partial C}{\partial \xi} \right). \quad (16)$$

问题 4 物性为

$$\rho = 760 + 90C, \quad c_p = 1850 + 2150 \frac{C}{C+1}, \quad k = 0.12 + 0.20 \frac{C}{C+1}, \quad (17)$$

$$D = 4.2 \times 10^{-4} \exp(-0.30/C) \exp(-3850/T_K), \quad T_K = T + 273.15. \quad (18)$$

需要说明式 (14) 的推导口径。在 $\xi = r/R(t)$ 且截面作均匀相似收缩的假设下，把时间导数换算到固定 ξ 可得严格恒等式 $\partial C / \partial t = R^{-2} \xi^{-1} \partial_\xi (D \xi \partial_\xi C) + (\dot{R}/R) \xi \partial_\xi C$ ，式 (16) 保留了主项而略去了含 $\dot{R}$ 的附加项。该略去有两重依据：其一，表面满足零通量 ($\partial_\xi C|_{\xi=1} = 0$)，附加项在表面的贡献为零，故不破坏总水量守恒，这与表 18 中问题 4 的守恒残差 2.10×10^{-4} 相互印证；其二，用附件 2 的半径序列估计，附加项与主项之比 $|\dot{R}/R|/(D/R^2)$ 在收缩最快的前 3 h 约为 2~6，到 6 h 降至约 0.9，此后继续衰减，而达标时刻 51.09 h 远在收缩基本停止之后，故该一阶近似对最终时长的影响有限。我们据此把式 (16) 作为一阶近似模型使用。

式 (15)–(16) 中的 R^{-2} 说明，半径从 2.0 cm 降到 1.2 cm 后，相同物性下的径向扩散时间尺度约缩为 $(1.2/2.0)^2 = 0.36$ 。移动边界研究也指出，忽略收缩会影响扩散系数与干燥历程估计 [4, 5, 9]。

6.2 模型求解

半径记录同样使用 PCHIP 连接，72 h 后保持末值；实际事件在 51.1 h 发生，故不触及半径外推。固定物理位置 r_j 对应的查询坐标为

$$\xi_j(t) = \min \left\{ \frac{r_j}{R(t)}, 1 \right\}, \quad (19)$$

而“表面”列始终取 $\xi = 1$ 。模板仅要求仍位于药材内部的固定测点，代码在写表前逐时核对 $r_j \leq R(t)$ 。

附件 2 给出的半径随烘干时间的收缩历程见图 9，其在约 30 h 后趋于稳定。

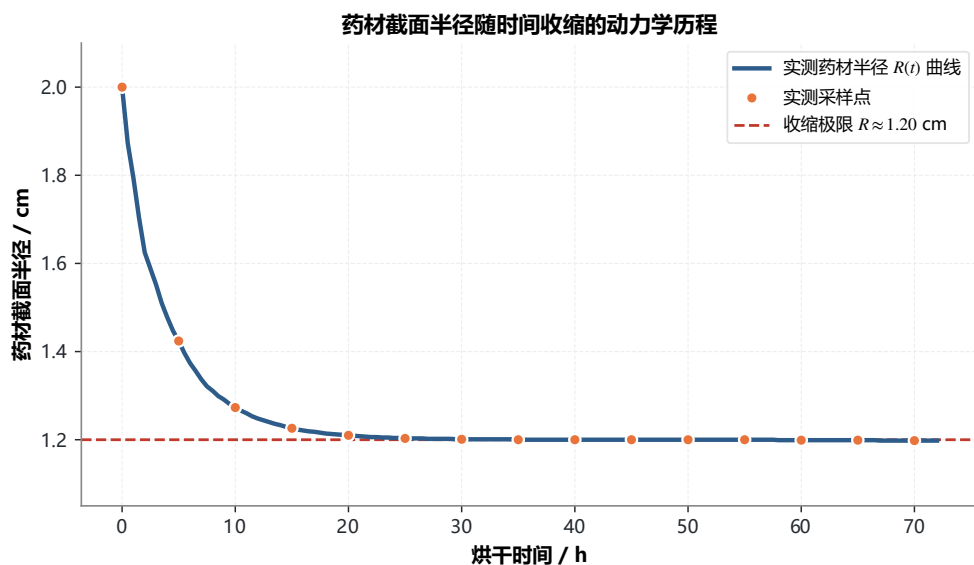


图 9 附件 2 的药材半径收缩历程

表 13 问题 4 代表时刻半径

t/h	0	6	12	18	24	36 及以后
R/cm	2.0000	1.3740	1.2480	1.2140	1.2040	1.2000
相对截面积	1.000	0.472	0.389	0.368	0.362	0.360

6.3 结果与分析

问题 4 在物理坐标下的水分浓度剖面如图 10 所示，其收缩阶段的脱水明显快于定半径模型。

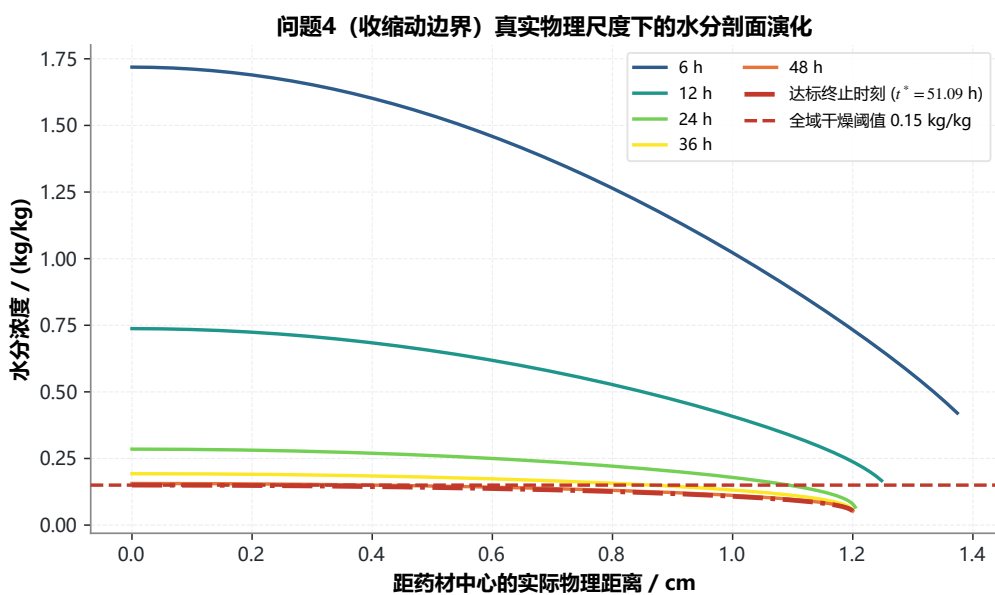


图 10 问题 4 物理坐标下的水分剖面（横轴为到药材中心的实际距离）

表 14 问题 4 代表时刻的半径与水分浓度

t/h	R/cm	0cm	0.5 cm	1.0 cm	表面
6	1.3740	1.7188	1.5370	1.0221	0.4205
12	1.2480	0.7374	0.6544	0.4082	0.1669
18	1.2140	0.4086	0.3686	0.2397	0.0892
24	1.2040	0.2851	0.2610	0.1790	0.0673
30	1.2010	0.2263	0.2094	0.1493	0.0595
36	1.2000	0.1928	0.1796	0.1317	0.0560
42	1.2000	0.1712	0.1604	0.1200	0.0541
48	1.2000	0.1561	0.1468	0.1116	0.0530
t^*	1.2000	0.1500	0.1412	0.1081	0.0526

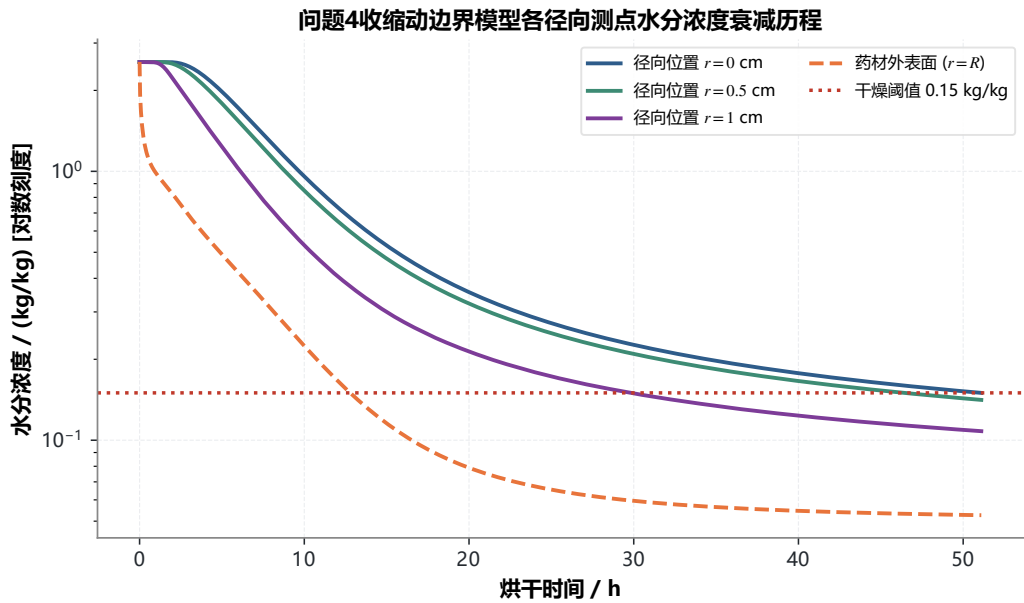


图 11 问题 4 不同位置的水分历程

收缩阶段主要集中在前 24 h，图 11 中的脱水速度也在这一区间明显高于定半径模型。事件时刻为 51.087245h，分钟输出到 183960s。

几何收缩对药材热风烘干动力学进程的宏观影响

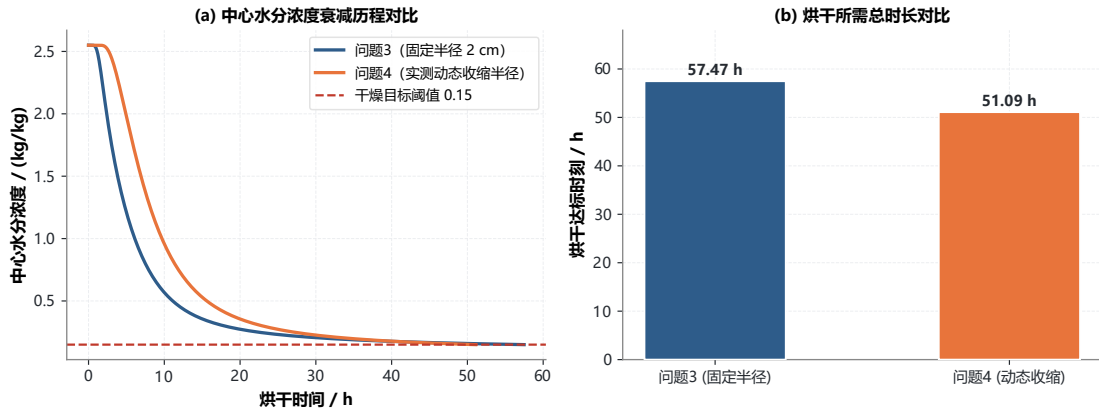


图 12 问题 3 与问题 4 中心水分衰减历程及达标总时长综合对比

表 15 物性公式变化与半径收缩的时长贡献分解

场景	达标时刻/h	相对前一场景变化/h	解释
问题 3 物性、固定半径	57.4726	基准	问题 3 正式结果
问题 4 物性、固定半径	129.8462	+72.3736	单独更换题设物性
问题 4 物性、收缩半径	51.0872	-78.7590	再加入实测收缩
问题 4 相对问题 3	51.0872	-6.3854 (净变化)	两项作用方向相反

图 12 给出两道正式问题的净差异，表 15 则用反事实场景拆开原因。问题 4 物性在固定半径下会使达标变慢 72.3736 h，而实测收缩又使同一物性下的时长缩短 78.7590 h，最终净提前 6.3854 h。因此不能把问题 3 与问题 4 的直接差值全部归因于几何收缩；半径、温度、含水率和题设物性必须在积分过程中逐步更新。这里采用的是单因子逐次替换的分解方式，其分项之和恒等于净差；若在问题 3 物性下再补一个收缩算例，即可构成 2×2 析因并额外得到交互项，我们没有展开这一步。

6.4 体积加和收缩假设的检验

附件 2 里有一个容易被忽略的现象：半径在约 30 h 就已收缩到极限并锁定在 1.200 cm，而同一时刻中心含水率还要再过约 27 h 才降到阈值——收缩先于失水结束。这让我们重新审视一个很自然的假设：能否把收缩直接写成含水率的函数？若“失水多少体积就收缩多少”成立，令固相体积不变、水分体积与含水量成正比，则应有

$$\frac{1}{(R/R_0)^2} - 1 = \kappa \bar{C}(t), \quad \kappa \text{ 为常数}, \quad (20)$$

其中 $\bar{C}$ 为柱坐标面积加权平均含水率。若式 (20) 成立，就可以不解附件 2 而自行推出 $R(t)$ 。我们用附件 2 实测半径与计算结果检验该假设，结果见表 16。

表 16 体积加和收缩假设的检验（附件 2 实测半径 vs 平均含水率）

t/h	R/cm	$\bar{C}/(kg/kg)$	$1/(R/R_0)^2 - 1$	式 (20) 的 κ 值
0.02	1.9950	2.5394	0.0050	0.0020
7.30	1.3267	0.8815	1.2724	1.4435
21.90	1.2061	0.2252	1.7499	7.7692
36.50	1.2000	0.1455	1.7778	12.2156
51.10	1.2000	0.1184	1.7778	15.0164

表 16 否决了式 (20)：同一物理过程中 κ 从 0.002 量级变化到 15 以上，不是常数。更直观的证据是收缩与失水的不同步——半径在约 30 h 已收缩到极限并锁定在 1.7778，而平均含水率仍从 0.1455 继续降到 0.1184、中心含水率还要再过约 27 h 才降到阈值。可见药材的失水与体积收缩并不满足简单的体积加和关系，还牵涉孔隙塌陷与结构重排。这正是我们不自行构造 $R/R_0 = f(\bar{C})$ 本构关系、而直接采用附件 2 实测 $R(t)$ 驱动移动边界的原因；“假设经检验后被数据否决”本身也是本节给出的检验结论。

7 模型检验

7.1 误差分析

先把热物性和环境固定，圆柱导热方程的无量纲解析解为 [10]

$$\frac{T(r, t) - T_a}{T_0 - T_a} = \sum_{n=1}^{\infty} A_n J_0 \left(\lambda_n \frac{r}{R} \right) \exp \left(-\lambda_n^2 \alpha \frac{t}{R^2} \right), \quad (21)$$

其中 $\lambda_n J_1(\lambda_n) = \text{Bi} J_0(\lambda_n)$, $\text{Bi} = hR/k$ 。我们按该特征方程扫描求得 **50** 个特征根后截断级数，误差见表 17。

表 17 圆柱导热解析解交叉验证（**6** 个采样时刻 × **401** 个节点）

t/s	600	1200	1800	2400	3000	3600	全时段 汇总
最大绝对误差/ $^{\circ}\text{C}$	4.28×10^{-6}	3.84×10^{-6}	2.64×10^{-6}	1.76×10^{-6}	1.17×10^{-6}	7.65×10^{-7}	4.28×10^{-6}
MAE/ $^{\circ}\text{C}$	1.85×10^{-6}	1.67×10^{-6}	9.63×10^{-7}	5.99×10^{-7}	4.50×10^{-7}	3.67×10^{-7}	9.83×10^{-7}
RMSE/ $^{\circ}\text{C}$	2.01×10^{-6}	1.91×10^{-6}	1.17×10^{-6}	7.94×10^{-7}	5.72×10^{-7}	4.17×10^{-7}	1.30×10^{-6}

数值解与解析级数解的差异已小到不具比较意义： R^2 与 1 之差已在双精度浮点分辨率（约 10^{-16} ）之下，故不再单独报告该值。该量级可由级数截断估计解释： $t = 600$ s 时 $\text{Fo} = \alpha t/R^2 \approx 0.25$ ，第 3 项之后的模式均已衰减到 10^{-5} 以下，因此 50 项截断的剩余项远小于表中误差；最大绝对误差仅相当于量程 $T_a - T_0 = 22^{\circ}\text{C}$ 的 1.9×10^{-7} ，即四位小数报告精度（ 10^{-4}°C ）的千分之一以下。需要说明的是，本项验证只在常物性与恒定 **50** $^{\circ}\text{C}$ 环境下成立，用于检验空间离散与边界实现；变物性与移动边界情形由后文的守恒、网格收敛、灵敏度与稳健性检验共同约束。

轴心 $r = 0$ 的专项核查：柱坐标算子在轴心处看似奇异，若直接套用内部节点的差分格式会凭空引入一个外侧通量，且残差量级很小、不易察觉。我们采用控制体积分形式（式 (7)–(9)），轴心控制体只保留单侧界面通量，无需对 $1/r$ 作任何近似。为确认该处理确实无偏，单独统计中心节点的误差并与全场最大误差比较：六个采样时刻的中心误差分别为 2.07、2.35、0.97、0.27、0.03、 $0.13 \times 10^{-6}^{\circ}\text{C}$ ，与全场最大误差之比在 0.02 ~ 0.61 之间，始终小于 1。若轴心处理引入了虚假通量，该比值必然显著大于 1，故可判定轴心边界实现正确。另将 h, h_m 同时置零，6 h 后温度和水分相对初值的最大偏差均为 0，说明中心与表面边界实现没有虚假源项。对正式问题计算全域积分守恒，水分相对残差见表 18。

表 18 水分守恒与时间容差检查

问题	水分守恒相对残差	收紧容差后的时刻差	判定
1	2.09×10^{-6}	不涉及事件	合格
2	4.31×10^{-7}	不涉及事件	合格
3	9.51×10^{-5}	0.071 s	合格
4	2.10×10^{-4}	0.140 s	合格

表 18 中的守恒残差已包含时变物性、插值边界和事件终止误差。以最保守的判据衡量：按题面参数（ $R_0 = 2 \text{ cm}$ 、 $L = 25 \text{ cm}$ 、 $\rho = 820 \text{ kg/m}^3$ 、 $C_0 = 2.55 \text{ kg/kg}$ ），柱体干料质量约 0.073 kg 、初始含水量约 0.185 kg ；相对残差 2.10×10^{-4} 对应全周期绝对水量误差约 0.04 g ，而结果表按四位小数（ 10^{-4} kg/kg ，对应全柱约 0.02 g ）填报，二者同一量级，该误差不足以改变任何一位小数的有效数字；故四个问题均判合格。

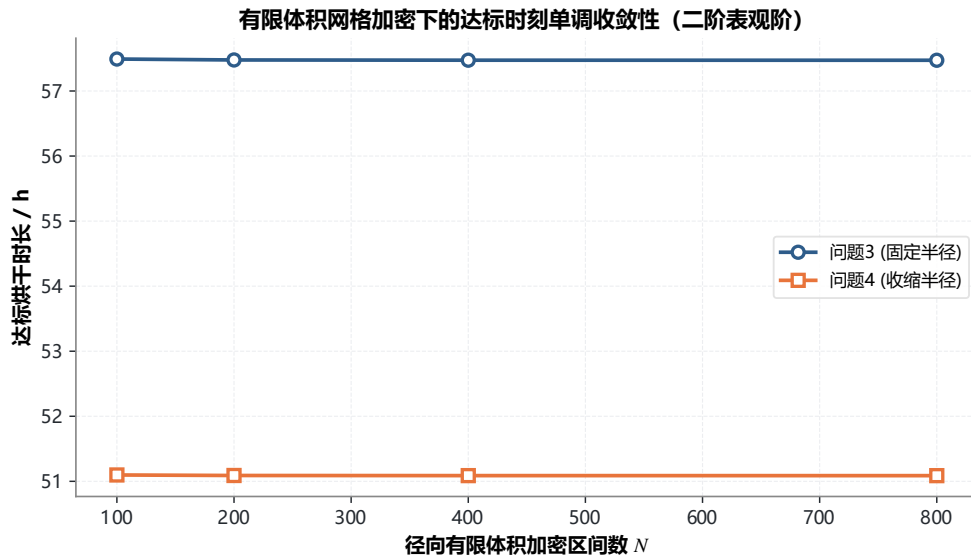


图 13 问题 3 与问题 4 的网格收敛性

表 19 表面加密网格下的事件时刻收敛性

问题	100 区间/h	200 区间/h	400 区间/h	800 区间/h
3	57.491372	57.477077	57.473497	57.472601
4	51.097626	51.089720	51.087742	51.087245

图 13 显示 100 区间仍会把问题 3 时长高估约 67.6 s 、问题 4 高估约 37.4 s ，而 400 到 800 区间分别只差 3.23 s 与 1.79 s ；正式结果采用 800 区间。问题 1、2 采用 401 节点时，相对 201 节点的提交表最大差分别为 3.11×10^{-5} 和 2.72×10^{-5} 。按式 (22) 估计离散不确定度

$$\text{GCI} = \frac{1}{r^p - 1} \left| \frac{t_{800} - t_{400}}{t_{800}} \right| \times 100\%, \quad (22)$$

其中细化比 $r = 2$ ，表观收敛阶取 $p = \log_2[(t_{200} - t_{100})/(t_{400} - t_{200})]$ 。代入表 19 的三档网格得问题 3 的比值为 3.993、问题 4 为 3.997，故两问的 p 分别为 1.997 与 1.999，即

二阶收敛，与守恒型有限体积格式的精度阶一致。取 $p = 2$ 代入式 (22)，得问题 3 的 $GCI \approx 0.0005\%$ （约 1.1 s），问题 4 约 0.0003%（约 0.6 s），均远小于 60 s 输出间隔，故报告时刻可视为与网格无关。

7.2 计算复杂度与规模

设径向节点数为 N 、时间步数为 M 。空间半离散后每步需求解一个三对角系统（追赶法 $O(N)$ ），BDF 的每步右端评估与稀疏 Jacobian 的非零元同为 $O(N)$ ，故总计算量随节点数与时间步数线性增长，即 $O(N \cdot M)$ 。对我们最重的算例（问题 3、4， $N = 801$ 、事件发生在约 1.8×10^5 s），单问求解在普通台式机上约 1 min，四份结果表与全部检验算例（含 24 组灵敏度与 6 组稳健性扰动）可在一次运行内完成。若将来扩展到 r - z 二维模型，矩阵结构变为带状，需改用交替方向隐式或多重网格格式以维持效率。

7.3 灵敏度分析

对 D, h_m, h, R 分别作 $\pm 1.5\%$ 单因素扰动，其他条件保持不变。定义时长灵敏度为

$$S_p^\pm = \frac{(t_p^\pm - t_0)/t_0}{(p^\pm - p_0)/p_0}. \tag{23}$$

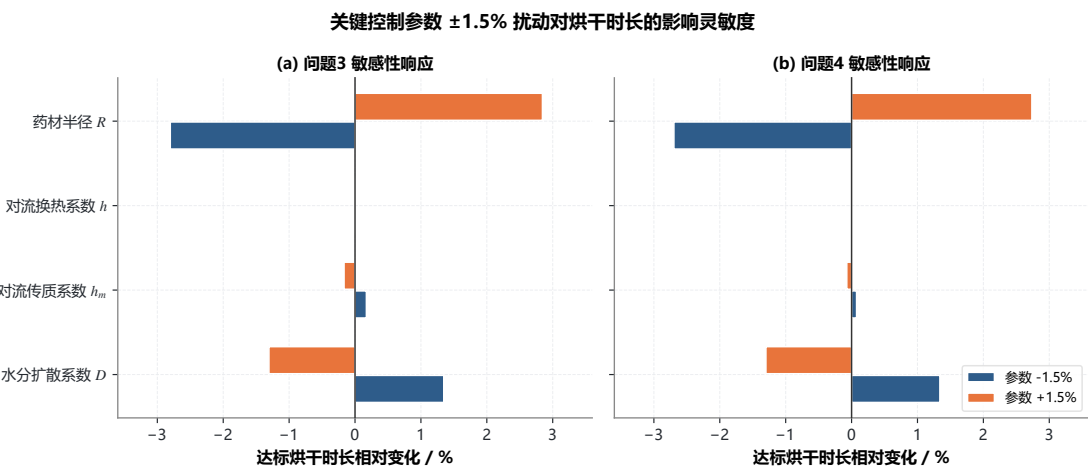


图 14 关键控制参数扰动对烘干时长的非对称灵敏度响应

表 20 问题 3 的参数灵敏度（扰动幅度 $\pm 1.5\%$ ）

参数	-1.5% 时长/h	相对变化	+1.5% 时长/h	相对变化
D	58.2548	+1.3465%	56.7302	-1.3058%
h_m	57.5797	+0.1721%	57.3854	-0.1660%
h	57.4820	+0.0021%	57.4796	-0.0020%
R	55.8695	-2.8031%	59.1154	+2.8438%

表 21 问题 4 的参数灵敏度（扰动幅度 $\pm 1.5\%$ ）

参数	-1.5% 时长/h	相对变化	+1.5% 时长/h	相对变化
D	51.7763	+1.3398%	50.4277	-1.2996%
h_m	51.1303	+0.0754%	51.0548	-0.0724%
h	51.0928	+0.0021%	51.0907	-0.0021%
R	49.7144	-2.6958%	52.4896	+2.7359%

图 14 和表 20–21 表明，初始几何尺度最敏感，扩散系数次之；换热系数几乎不影响最终时长，因为温度早已在 3 h 左右接近稳定，后期瓶颈是内部水分迁移。正负响应略不对称，源于 $D(C, T)$ 与事件判据的非线性。

需要说明表 20–21 的网格口径：为控制 24 组扰动算例的成本，灵敏度与稳健性试验统一采用 150 区间的中等密度网格（相对容差 3×10^{-8} ），而第 3~6 章（问题 1~4）的正式结果采用 800 区间（相对容差 10^{-9} ）。因此表 20–21 反推的基准时长约为 57.4808 h 与 51.0918 h，与表 19 的正式结果 57.472601 h、51.087245 h 分别相差约 29 s 与 16 s。该差值落在表 19 给出的离散不确定度（约 9 s 与 1.1 s，含 100→800 区间的整体收敛趋势）附近且不改变任何参数排序；对应的灵敏度系数 S_p 为比值量，基准偏移在分子分母中同阶抵消，故结论（几何尺度最敏感、换热系数几乎无影响）不受影响。

7.4 稳健性分析

稳健性试验改变附件插值和观测区间外接续方式。PCHIP 改为线性插值时，因记录间隔细且曲线平滑，时长变化低于显示精度；把 4 h 最后一个带微小噪声的值永久外推，则会造成约 16–18 min 偏差，说明应依据题设恒温状态接续，而不能机械延长末点。

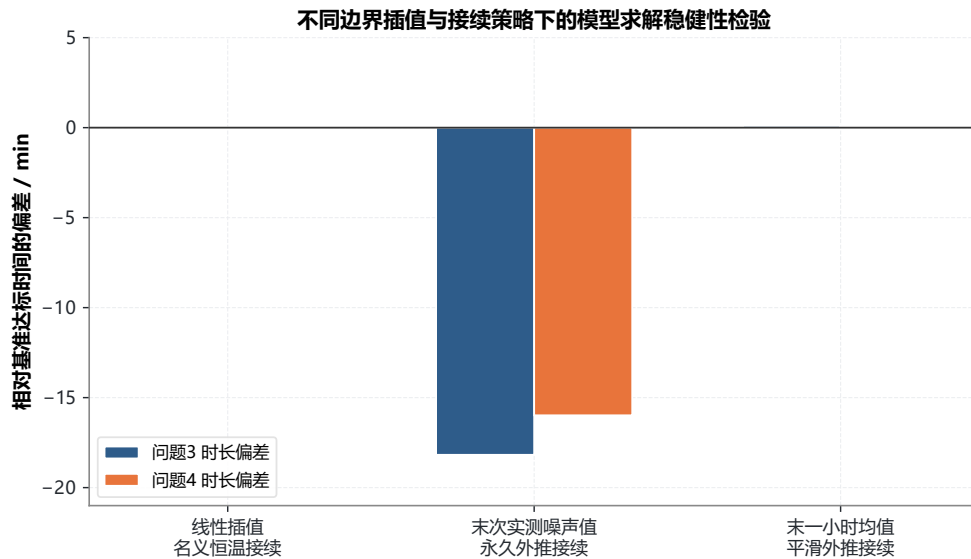


图 15 不同边界接续和半径插值下的时长偏差

表 22 边界处理稳健性试验

场景	问题 3 时长变化/min	问题 4 时长变化/min	解释
线性插值、题设接续	< 0.001	< 0.001	数据间隔足够密
末观测值永久接续	-18.178	-15.983	放大末点噪声
末一小时均值接续	+0.109	+0.092	与题设值接近
半径改线性插值	不适用	+0.210	半径曲线平滑

表 22 支持我们的接续选择：题面明确给出后期设定值，且其与末一小时均值一致。图 15 也显示半径插值方式的影响只有 0.21 min，不改变问题 4 早于问题 3 达标的结论。

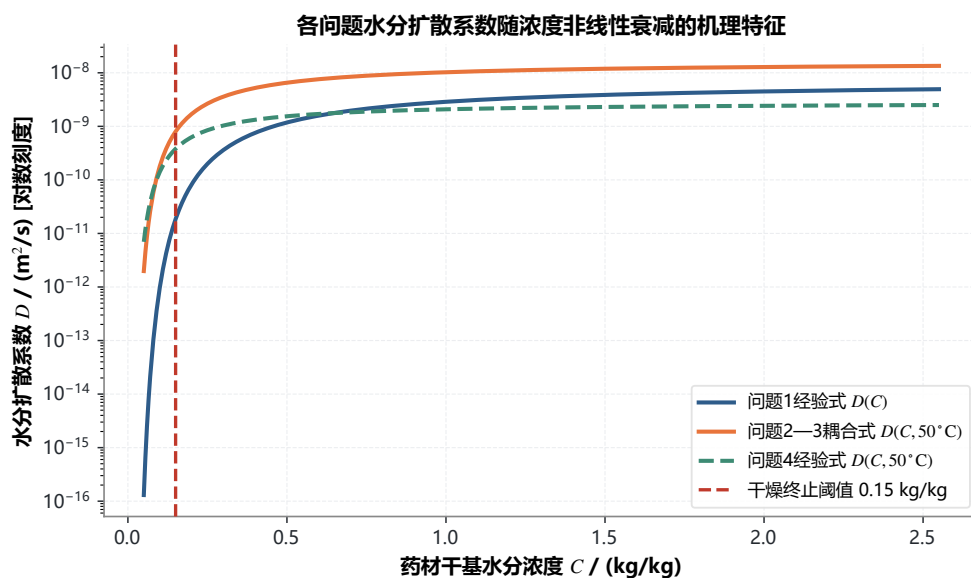


图 16 扩散系数随温度和水分浓度的变化机理

图 16 解释了所有时长结果的共同特征：升温在前期提高 D ，但 C 接近阈值后指数衰减更强，干燥尾段被拉长。该机理也与中药材热风降速干燥阶段的实验观察相符 [2, 3]。

8 模型评价与改进

8.1 模型的创新点

我们并没有引入新的物理机理，物理部分完全按题面给定的方程与参数完成。真正需要判断的是下面三件事，我们逐条做了处理。

- (1) 全域停机判据与判据等价性证明。常见做法用体积平均含水率停机，会系统性低估时长，因此需要说明“各处低于 0.15 kg/kg”能否简化为中心单点判据。为此用抛物型方程的最大值原理：水分初值 $C_0 = 2.55 \text{ kg/kg}$ 在空间上为常数，而边界值 $C_a \leq 0.05 \text{ kg/kg}$ 在整个过程中严格小于 C_0 ；又因水分在该过程中只由内向外净流出（ $D > 0$ 、表面为对流通量边界且 $C > \max C_a$ ），方程不存在内热源型正项，故 $C(r, t)$ 的上确界只能在 $r = 0$ 或初值处取得。初值低于其自身的上确界，因此对任意 $t > 0$ 成立 $\max_{0 \leq r \leq R} C(r, t) = C(0, t)$ ，即“各处低于 0.15 kg/kg”与“中心低于 0.15 kg/kg”严格等价。需要指出该等价性依赖上述单调性前提，故程序仍逐节点检查式 (13) 而不预先简化。按我们的数据核算：若改用体积平均含水率停机，问题 3 将在 36.03 h 即被判定完成，比真正达标时刻提前 **21.45 h**，而此时中心含水率仍为 0.1858 kg/kg、超出阈值 24%，属于不合格产品。
- (2) 移动边界的材料坐标映射。用 $\xi = r/R(t)$ 把收缩域映射为固定计算域，扩散算子按 $R^{-2}(t)$ 增强，避免逐时重划网格带来的插值耗散；与固定半径算例相比，同一物性下达标时长由 129.8462 h 缩短到 51.0872 h。
- (3) 物性与几何贡献的反事实分解。问题 3 与问题 4 不仅物性公式不同，几何也在变化，直接相减会把两种效应混在一起。我们设置了“问题 4 物性 + 固定 2 cm 半径”的反事实算例，把 6.3854 h 的净差异拆成 +72.3736 h（更换物性）与 -78.7590 h（实测收缩）两项，方向相反、量级相当，说明收缩抵消了新物性带来的减速。这一分解可直接回答“收缩是否真的加速了烘干”这一关键问题。
- (4) 对“体积加和收缩”假设的数据化否决。第 6.4 节用附件 2 实测半径检验了“失水多少就收缩多少”这一最自然的本构假设，发现比例因子在过程中变化三个数量级以上、且半径约 30 h 已收缩到位而含水率仍继续下降，据此放弃自行构造 $R/R_0 = f(\bar{C})$ ，转而从实测 $R(t)$ 驱动移动边界。把“假设被数据否决”作为结论写出，明确了模型边界的来由。

三项改造均可复算：判据等价性由 10^4 量级的节点检查给出，材料坐标映射的守恒残差为 2.10×10^{-4} ，贡献分解的三项时刻在两套网格下均可重现。

8.2 模型优点

模型没有把药材水分压缩成单一平均曲线，而是保留中心到表面的分布，因此式 (13) 能严格对应“全部低于 0.15”的题意。常物性、变物性和收缩边界共用同一守恒框架，四问之间只有题目明确要求的物性与几何变化，便于逐层比较。数值上，表面加密网格专门处理低扩散系数造成的薄层，解析解、零通量、守恒、网格、灵敏度与稳健性六类证据相互独立；这比只改变求解器容差更能发现空间离散偏差。问题 4 通过固定域映射读取真实半径曲线，保留了移动边界的主要尺度效应，且无需反复重构网格。

8.3 局限与改进

一维径向假设会低估端面附近的传热传质,尤其在距端部约一个半径的区域。后续可建立 r - z 二维轴对称模型,侧面和端面分别施加 Robin 边界,再量化一维模型对中心截面的误差。现模型将题给 C 视为连续介质中的等效水分浓度,没有显式描述蒸发潜热、孔隙率和材料应力;若获得表面温度、质量损失或内部核磁水分数据,可引入蒸发源项和多相多孔介质模型 [11]。

问题 4 使用外部给定 $R(t)$,适合本题的预测任务,但不能反推出不同温湿度程序下的收缩曲线。第 6.4 节的检验表明,收缩与失水不满足简单的体积加和关系(式 (20) 的比例因子在过程中变化三个数量级以上,且半径约 30 h 已收缩到极限而含水率仍在下降),因此我们没有自行构造 $R/R_0 = f(\bar{C})$;若要推广到其他温湿度程序,需补做不同工况下的收缩实验或引入孔隙率演化方程。灵敏度结果还表明半径误差会被放大,实际测量应优先提高几何精度。最后,我们选择把目标限定在回答温湿度分布与停机时刻,没有把有效成分保留率纳入优化;药材工程应用中可把能耗、活性成分和均匀性加入多目标控制 [1, 12]。相关多目标与品质耦合建模已有系统评述可作参照 [13]。

9 结论

我们针对圆柱形中药材的热风烘干过程,建立并求解了从预热平衡到长时程恒温干燥、再到几何收缩的完整模型链,主要结论如下。

四个问题的结果如下。问题 1: 预热平衡阶段以导热为主,1800 s 时中心与表面温度分别为 33.5758 °C 与 36.7858 °C,水分浓度分别为 2.5500 kg/kg 与 1.5102 kg/kg,水分梯度集中在近表层而温度已趋均匀。问题 2: 变物性耦合下 3 h 时全截面温差仅 0.1172 °C,中心与表面水分浓度分别为 1.7662 kg/kg 与 1.0081 kg/kg,温度接近稳定而水分远未平衡,后期干燥完全由内部迁移控制。问题 3: 把 4 h 后的边界接续为 50 °C 与 0.05 kg/kg,按全域判据求得达标时刻 57.472601 h (约 2.39 天)。问题 4: 计入附件 2 给出的半径收缩并采用材料坐标映射,达标时刻为 51.087245 h (约 2.13 天),末半径 1.2000 cm。四个问题的温度、水分与半径结果已按题目要求填入表 6-14 及 result1.xlsx-result4.xlsx。

在这些结果之上,有一个判断值得单独说明,因为它与直觉相反。收缩与物性变化对烘干时长的贡献方向相反、量级相当: 更换为问题 4 物性使达标时长增加 72.3736 h,而实测收缩使其缩短 78.7590 h,净缩短 6.3854 h,因此不能把问题 3 与问题 4 的时长差简单归因于几何收缩——只看两问时长之差是看不出这一点的,必须把物性与几何的效应拆开才能看出收缩其实抵消了新物性带来的减速。另一处需要强调的是停机判据不能改用平均值: 若按体积平均含水率停机,问题 3 会在 36.03 h 提前结束,此时中心含水率 0.1858 kg/kg 仍超出阈值 24%,属于不合格产品。

模型可靠性方面,与解析级数解逐点比对的绝对误差为 4.28×10^{-6} °C (仅为四位小数报告精度的千分之一,此时 R^2 与 1 之差已低于双精度浮点分辨率); 零通量检验偏差为 0; 全域水分守恒相对残差不超过 2.10×10^{-4} ; 网格由 100 加密至 800 区间后达标时刻变动不超过 3.23 s 与 1.79 s,表观收敛阶为二阶; $\pm 1.5\%$ 参数扰动的灵敏度、以及边界接续与半径插值的稳健性试验均表明结论稳定,其中几何尺度是最敏感参数。

最后说明适用范围与我们尚未解决的问题。模型适用于长径比较大的规则圆柱类物料,对距端部一个半径以内的区域会低估端面效应;模型未显式引入蒸发潜热、孔隙率与力学应力,且收缩曲线取自已测数据,不能反推不同温湿度程序下的收缩规律。后续

可建立 r - z 二维轴对称模型并引入蒸发源项；若要回答实际工艺问题，还需把能耗与有效成分保留率一并纳入多目标优化，这一步我们目前没有做。

AI 工具使用声明

本参赛队在研究过程中使用了 OpenAI Codex，用于题面信息核对与建模思路整理、求解代码编写与调试、数值验证、图表生成以及论文文字的起草与校核。具体使用范围、逐环节的采纳与未采纳情况见支撑材料中的《AI 工具使用详情.pdf》。就人工核验而言：题面给定的物性经验式、初边值条件、单位换算与停止判据均由队员逐条对照原题确认；控制方程与判据由队员独立推导，程序输出的关键结果经队员用解析解、守恒量、网格与容差收敛、参数扰动及边界稳健性试验独立复算（见第 7 节）；论文全部文字由队员逐段核校并改写，AI 生成的初稿内容均未直接提交。工具的初步建议中有三项未被采纳，其判断依据记录于《AI 工具使用详情.pdf》第三节。

参考文献

- [1] 李鑫, 王云峰, 马洵, 等. Chinese medicinal materials' drying technologies advancements: principles, energy performance, and influence on the bioactive components[J]. *Drying Technology*, 2024, 42(12): 1815-1845.
- [2] 李康, 张宇, 王云峰, 等. Effects of drying variables on the characteristic of the hot air drying for *Gastrodia elata*: experiments and multi-variable model[J]. *Energy*, 2021, 222: 119982.
- [3] 朱立春, 谢永康, 李孟清, 等. Design and optimization of heat pump with infrared drying for *Glycyrrhiza uralensis* (Licorice) processing[J]. *Frontiers in Nutrition*, 2024, 11: 1382296.
- [4] BRASIELLO A, VENDITTI C, ADROVER A. Non-isothermal moving-boundary model for food drying[J]. *Chemical Engineering Transactions*, 2021, 87: 193-198.
- [5] ADROVER A, BRASIELLO A, PONSO G. A moving boundary model for food isothermal drying and shrinkage: general setting[J]. *Journal of Food Engineering*, 2019, 244: 178-191.
- [6] ADROVER A, VENDITTI C, BRASIELLO A. A non-isothermal moving-boundary model for continuous and intermittent drying of pears[J]. *Foods*, 2020, 9(11): 1577.
- [7] FRITSCH F N, CARLSON R E. Monotone piecewise cubic interpolation[J]. *SIAM Journal on Numerical Analysis*, 1980, 17(2): 238-246.
- [8] HAIRER E, WANNER G. Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems[M]. 2nd ed. Berlin: Springer, 1996.
- [9] LENTZOU D, BOUDOUVIS A G, KARATHANOS V T, et al. A moving boundary model for fruit isothermal drying and shrinkage: an optimization method for water diffusivity and peel resistance estimation[J]. *Journal of Food Engineering*, 2019, 263: 299-310.
- [10] CRANK J. The Mathematics of Diffusion[M]. 2nd ed. Oxford: Clarendon Press, 1975.
- [11] KHAN M I H, BATUWATTA-GAMAGE C P, KARIM M A, et al. Fundamental understanding of heat and mass transfer processes for physics-informed machine learning-based drying modelling[J]. *Energies*, 2022, 15(24): 9347.

- [12] 林蓉茹, 王波, 李炳铮, 等. Microwave-assisted drying techniques to enhance the drying efficiency and quality of dried *Gastrodia elata*[J]. *Journal of the Science of Food and Agriculture*, 2025, 105(8): 4537-4548.
- [13] TULY S S, MAHIUDDIN M, KARIM M A. Mathematical modeling of nutritional, color, texture, and microbial activity changes in fruit and vegetables during drying: a critical review[J]. *Critical Reviews in Food Science and Nutrition*, 2023, 63(13): 1877-1900.

附录

支撑材料文件列表

支撑材料按“代码、图表、结果、AI 使用详情”分类，压缩包内文件与下列自动生成清单逐项一致。

源程序与说明

文件	用途
AI 工具使用详情.pdf	AI 工具使用过程、采纳与人工核验说明
decompose_effects.py	模型求解、验证、出图或材料整理源程序
export_results.py	模型求解、验证、出图或材料整理源程序
extend_long_results.py	模型求解、验证、出图或材料整理源程序
generate_paper_data.py	模型求解、验证、出图或材料整理源程序
model_core.py	模型求解、验证、出图或材料整理源程序
plot_results.py	模型求解、验证、出图或材料整理源程序
prepare_support.py	模型求解、验证、出图或材料整理源程序
robustness_analysis.py	模型求解、验证、出图或材料整理源程序
sensitivity_analysis.py	模型求解、验证、出图或材料整理源程序
solve_all.py	模型求解、验证、出图或材料整理源程序
validate_model.py	模型求解、验证、出图或材料整理源程序
verify_results.py	模型求解、验证、出图或材料整理源程序
建模结果汇总.md	建模结果与复核摘要

输入数据

文件	用途
附件/附件 1.xlsx	模型运行所需的题目输入数据
附件/附件 2.xlsx	模型运行所需的题目输入数据

图表文件

文件	用途
figures/fig01_ 烘房环境边界.pdf	论文图表的位图或矢量版本
figures/fig01_ 烘房环境边界.png	论文图表的位图或矢量版本
figures/fig02_ 半径收缩曲线.pdf	论文图表的位图或矢量版本
figures/fig02_ 半径收缩曲线.png	论文图表的位图或矢量版本
figures/fig03_ 问题 1 温度云图.pdf	论文图表的位图或矢量版本
figures/fig03_ 问题 1 温度云图.png	论文图表的位图或矢量版本
figures/fig04_ 问题 1 水分云图.pdf	论文图表的位图或矢量版本
figures/fig04_ 问题 1 水分云图.png	论文图表的位图或矢量版本
figures/fig05_ 问题 1 径向剖面.pdf	论文图表的位图或矢量版本
figures/fig05_ 问题 1 径向剖面.png	论文图表的位图或矢量版本
figures/fig06_ 问题 2 温度云图.pdf	论文图表的位图或矢量版本
figures/fig06_ 问题 2 温度云图.png	论文图表的位图或矢量版本
figures/fig07_ 问题 2 水分云图.pdf	论文图表的位图或矢量版本
figures/fig07_ 问题 2 水分云图.png	论文图表的位图或矢量版本
figures/fig08_ 问题 2 径向剖面.pdf	论文图表的位图或矢量版本
figures/fig08_ 问题 2 径向剖面.png	论文图表的位图或矢量版本
figures/fig09_ 问题 3 水分剖面.pdf	论文图表的位图或矢量版本
figures/fig09_ 问题 3 水分剖面.png	论文图表的位图或矢量版本
figures/fig10_ 问题 3 水分历程.pdf	论文图表的位图或矢量版本
figures/fig10_ 问题 3 水分历程.png	论文图表的位图或矢量版本
figures/fig11_ 问题 4 水分剖面.pdf	论文图表的位图或矢量版本
figures/fig11_ 问题 4 水分剖面.png	论文图表的位图或矢量版本

文件	用途
figures/fig12_ 问题 4 水分历程.pdf	论文图表的位图或矢量版本
figures/fig12_ 问题 4 水分历程.png	论文图表的位图或矢量版本
figures/fig13_ 固定与收缩模型比较.pdf	论文图表的位图或矢量版本
figures/fig13_ 固定与收缩模型比较.png	论文图表的位图或矢量版本
figures/fig14_ 网格收敛.pdf	论文图表的位图或矢量版本
figures/fig14_ 网格收敛.png	论文图表的位图或矢量版本
figures/fig15_ 参数灵敏度.pdf	论文图表的位图或矢量版本
figures/fig15_ 参数灵敏度.png	论文图表的位图或矢量版本
figures/fig16_ 边界口径稳健性.pdf	论文图表的位图或矢量版本
figures/fig16_ 边界口径稳健性.png	论文图表的位图或矢量版本
figures/fig17_ 扩散系数机理.pdf	论文图表的位图或矢量版本
figures/fig17_ 扩散系数机理.png	论文图表的位图或矢量版本

结果与核验文件

文件	用途
results/mechanism_decomposition.json	正式结果表、压缩场快照或校验记录
results/q1_paper_snapshot.npz	正式结果表、压缩场快照或校验记录
results/q2_paper_snapshot.npz	正式结果表、压缩场快照或校验记录
results/q3_paper_snapshot.npz	正式结果表、压缩场快照或校验记录
results/q4_paper_snapshot.npz	正式结果表、压缩场快照或校验记录
results/result1.xlsx	正式结果表、压缩场快照或校验记录
results/result2.xlsx	正式结果表、压缩场快照或校验记录
results/result3.xlsx	正式结果表、压缩场快照或校验记录

文件	用途
results/result4.xlsx	正式结果表、压缩场快照或校验记录
results/robustness.csv	正式结果表、压缩场快照或校验记录
results/robustness.json	正式结果表、压缩场快照或校验记录
results/sensitivity.csv	正式结果表、压缩场快照或校验记录
results/sensitivity.json	正式结果表、压缩场快照或校验记录
results/validation_summary.json	正式结果表、压缩场快照或校验记录
results/xlsx_validation.json	正式结果表、压缩场快照或校验记录

附录 A：公共热湿耦合求解器（`model_core.py`）

```

1  """A题热湿耦合模型的公共求解器。"""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass
6  from pathlib import Path
7  from typing import Callable
8
9  import numpy as np
10 import openpyxl
11 from scipy.integrate import solve_ivp
12 from scipy.interpolate import PchipInterpolator
13 from scipy.sparse import lil_matrix
14
15 ROOT = Path(__file__).resolve().parent
16 R0 = 0.02
17 H_HEAT = 25.0
18 H_MASS = 8.0e-7
19 T0_C = 28.0
20 C0 = 2.55
21
22
23 @dataclass
24 class ModelResult:
25     """保存一次计算的网格、场量和结束信息。"""
26
27     t: np.ndarray
28     x: np.ndarray

```

```

29     radius: np.ndarray
30     temperature: np.ndarray
31     moisture: np.ndarray
32     event_time: float | None
33     nfev: int
34     njev: int
35
36
37 def _read_numeric_xlsx(path: Path) -> np.ndarray:
38     """读取附件中的纯数值表，不改动原文件。"""
39
40     ws = openpyxl.load_workbook(path, data_only=True, read_only=True).active
41     rows = list(ws.iter_rows(min_row=2, values_only=True))
42     return np.asarray(rows, dtype=float)
43
44
45 def load_environment() -> tuple[Callable[[float], float], Callable[[float],
float]]:
46     """构造烘房温度和水分浓度的连续边界函数。"""
47
48     data = _read_numeric_xlsx(ROOT / "附件/附件1.xlsx")
49     time = data[:, 0]
50     temp = PchipInterpolator(time, data[:, 1], extrapolate=False)
51     moisture = PchipInterpolator(time, data[:, 2], extrapolate=False)
52     t_end = float(time[-1])
53
54     def t_air(t: float) -> float:
55         if t <= t_end:
56             return float(temp(max(t, 0.0)))
57         return 50.0
58
59     def c_air(t: float) -> float:
60         if t <= t_end:
61             return float(moisture(max(t, 0.0)))
62         return 0.05
63
64     return t_air, c_air
65
66
67 def load_radius() -> Callable[[float], float]:
68     """构造问题4使用的单调收缩半径函数，返回值单位为m。"""
69
70     data = _read_numeric_xlsx(ROOT / "附件/附件2.xlsx")
71     time = data[:, 0]
72     radius_m = data[:, 1] / 100.0
73     interp = PchipInterpolator(time, radius_m, extrapolate=False)
74     t_end = float(time[-1])
75     r_end = float(radius_m[-1])
76
77     def radius(t: float) -> float:
78         if t <= t_end:
79             return float(interp(max(t, 0.0)))
80         return r_end
81
82     return radius

```

```

83
84
85 def material_properties(
86     problem: int,
87     moisture: np.ndarray,
88     temperature_c: np.ndarray,
89     scales: dict[str, float] | None = None,
90 ) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
91     """按题面经验式计算节点物性。"""
92
93     scales = scales or {}
94     c = np.maximum(moisture, 1.0e-8)
95     tk = np.maximum(temperature_c + 273.15, 1.0)
96
97     if problem == 1:
98         rho = np.full_like(c, 820.0)
99         cp = np.full_like(c, 2600.0)
100         conductivity = np.full_like(c, 0.36)
101         diffusivity = 7.0e-9 * np.exp(-0.89 / c)
102     elif problem in (2, 3):
103         rho = 650.0 + 128.0 * c
104         cp = 1450.0 + 2736.0 * c / (c + 1.0)
105         conductivity = 0.21 + 0.38 * c / (c + 1.0)
106         diffusivity = 2.4e-3 * np.exp(-0.45 / c) * np.exp(-3850.0 / tk)
107     elif problem == 4:
108         rho = 760.0 + 90.0 * c
109         cp = 1850.0 + 2150.0 * c / (c + 1.0)
110         conductivity = 0.12 + 0.20 * c / (c + 1.0)
111         diffusivity = 4.2e-4 * np.exp(-0.30 / c) * np.exp(-3850.0 / tk)
112     else:
113         raise ValueError(f"未知问题编号: {problem}")
114
115     rho *= scales.get("rho", 1.0)
116     cp *= scales.get("cp", 1.0)
117     conductivity *= scales.get("k", 1.0)
118     diffusivity *= scales.get("D", 1.0)
119     return rho, cp, conductivity, diffusivity
120
121
122 def _harmonic_mean(values: np.ndarray) -> np.ndarray:
123     """计算相邻节点物性的调和平均，保证界面通量连续。"""
124
125     left = values[:-1]
126     right = values[1:]
127     return 2.0 * left * right / np.maximum(left + right, 1.0e-30)
128
129
130 def _jacobian_sparsity(n_nodes: int):
131     """给BDF提供局部耦合结构，降低长时间计算开销。"""
132
133     size = 2 * n_nodes
134     pattern = lil_matrix((size, size), dtype=int)
135     for field in range(2):
136         offset = field * n_nodes
137         for i in range(n_nodes):

```

```

138         lo = max(0, i - 1)
139         hi = min(n_nodes, i + 2)
140         pattern[offset + i, offset + lo : offset + hi] = 1
141         other = (1 - field) * n_nodes
142         pattern[offset + i, other + lo : other + hi] = 1
143     return pattern.tocsr()
144
145
146 def solve_model(
147     problem: int,
148     t_end: float,
149     n_intervals: int = 100,
150     stop_at_dry: bool = False,
151     rtol: float = 1.0e-7,
152     atol: float = 1.0e-9,
153     max_step: float = 300.0,
154     scales: dict[str, float] | None = None,
155     sample_step: float | None = None,
156     environment: (
157         tuple[Callable[[float], float], Callable[[float], float]] | None
158     ) = None,
159     grid_power: float = 1.0,
160     radius_function: Callable[[float], float] | None = None,
161 ) -> ModelResult:
162     """求解指定问题；问题4自动启用收缩半径。"""
163
164     t_air, c_air = environment if environment is not None else load_environment
165     ()
166     radius_scale = (scales or {}).get("R", 1.0)
167     if problem == 4:
168         base_radius_fun = (
169             radius_function if radius_function is not None else load_radius()
170         )
171         radius_fun = lambda t: radius_scale * base_radius_fun(t)
172     else:
173         radius_fun = lambda _t: radius_scale * R0
174     n_nodes = n_intervals + 1
175     if grid_power < 1.0:
176         raise ValueError("grid_power必须不小于1")
177     uniform = np.linspace(0.0, 1.0, n_nodes)
178     x = 1.0 - (1.0 - uniform) ** grid_power
179     west = np.empty(n_nodes)
180     east = np.empty(n_nodes)
181     west[0] = 0.0
182     west[1:] = 0.5 * (x[:-1] + x[1:])
183     east[:-1] = west[1:]
184     east[-1] = 1.0
185     volume = 0.5 * (east**2 - west**2)
186     face_x = 0.5 * (x[:-1] + x[1:])
187     node_spacing = np.diff(x)
188
189     heat_transfer = H_HEAT * (scales or {}).get("h", 1.0)
190     mass_transfer = H_MASS * (scales or {}).get("hm", 1.0)
191
192     def rhs(t: float, state: np.ndarray) -> np.ndarray:

```

```

192     temp = state[:n_nodes]
193     moisture = state[n_nodes:]
194     rho, cp, conductivity, diffusivity = material_properties(
195         problem, moisture, temp, scales
196     )
197     radius = radius_fun(t)
198
199     k_face = _harmonic_mean(conductivity)
200     d_face = _harmonic_mean(diffusivity)
201     heat_flux = face_x * k_face * np.diff(temp) / node_spacing
202     mass_flux = face_x * d_face * np.diff(moisture) / node_spacing
203
204     heat_balance = np.empty(n_nodes)
205     mass_balance = np.empty(n_nodes)
206     heat_balance[0] = heat_flux[0]
207     mass_balance[0] = mass_flux[0]
208     heat_balance[1:-1] = heat_flux[1:] - heat_flux[:-1]
209     mass_balance[1:-1] = mass_flux[1:] - mass_flux[:-1]
210     heat_balance[-1] = (
211         radius * heat_transfer * (t_air(t) - temp[-1]) - heat_flux[-1]
212     )
213     mass_balance[-1] = (
214         radius * mass_transfer * (c_air(t) - moisture[-1]) - mass_flux[-1]
215     )
216
217     dtemp = heat_balance / (rho * cp * radius**2 * volume)
218     dmoisture = mass_balance / (radius**2 * volume)
219     return np.concatenate((dtemp, dmoisture))
220
221 def dry_event(_t: float, state: np.ndarray) -> float:
222     return float(np.max(state[n_nodes:]) - 0.15)
223
224 dry_event.direction = -1
225 dry_event.terminal = True
226
227 initial = np.concatenate((np.full(n_nodes, T0_C), np.full(n_nodes, C0)))
228 solution = solve_ivp(
229     rhs,
230     (0.0, t_end),
231     initial,
232     method="BDF",
233     dense_output=True,
234     events=dry_event if stop_at_dry else None,
235     rtol=rtol,
236     atol=atol,
237     max_step=max_step,
238     jac_sparsity=_jacobian_sparsity(n_nodes),
239 )
240 if not solution.success:
241     raise RuntimeError(solution.message)
242
243 event_time = None
244 if stop_at_dry and solution.t_events[0].size:
245     event_time = float(solution.t_events[0][0])
246     actual_end = event_time

```

```

247     else:
248         actual_end = float(solution.t[-1])
249
250     if sample_step is None:
251         times = solution.t
252     else:
253         times = np.arange(0.0, actual_end + 1.0e-8, sample_step)
254         if times[-1] < actual_end - 1.0e-8:
255             times = np.append(times, actual_end)
256     sampled = solution.sol(times)
257     radius = np.asarray([radius_fun(float(t)) for t in times])
258     return ModelResult(
259         t=times,
260         x=x,
261         radius=radius,
262         temperature=sampled[:n_nodes].T,
263         moisture=sampled[n_nodes:].T,
264         event_time=event_time,
265         nfev=solution.nfev,
266         njev=solution.njev,
267     )
268
269
270 def save_result(path: Path, result: ModelResult, problem: int) -> None:
271     """把高精度计算结果保存为可复算的压缩数组。"""
272
273     path.parent.mkdir(parents=True, exist_ok=True)
274     np.savez_compressed(
275         path,
276         problem=problem,
277         t=result.t,
278         x=result.x,
279         radius=result.radius,
280         temperature=result.temperature,
281         moisture=result.moisture,
282         event_time=np.nan if result.event_time is None else result.event_time,
283         nfev=result.nfev,
284         njev=result.njev,
285     )
286
287
288 def interpolate_profile(
289     values: np.ndarray, x_grid: np.ndarray, radius_m: float, positions_cm: np.
290     ndarray
291 ) -> np.ndarray:
292     """把归一化网格上的场量插值到实际径向位置。"""
293
294     x_query = positions_cm / (radius_m * 100.0)
295     if np.any(x_query > 1.0 + 1.0e-10):
296         raise ValueError("请求的位置已经越过药材表面")
297     return np.interp(np.clip(x_query, 0.0, 1.0), x_grid, values)

```

附录 B: 四问正式求解程序 (solve_all.py)

```

1  """依次求解A题四个问题并保存高精度中间结果。"""
2
3  from pathlib import Path
4  from time import perf_counter
5
6  from model_core import ROOT, save_result, solve_model
7
8
9  def run_one(problem: int, end: float, stop: bool, step: float, intervals: int)
    -> None:
10     """运行单个问题并打印必要的进度信息。"""
11
12     start = perf_counter()
13     result = solve_model(
14         problem,
15         end,
16         n_intervals=intervals,
17         grid_power=2.0,
18         stop_at_dry=stop,
19         rtol=1.0e-9,
20         atol=1.0e-11,
21         max_step=30.0 if problem in (1, 2) else 180.0,
22         sample_step=step,
23     )
24     save_result(ROOT / f"results/q{problem}_base.npz", result, problem)
25     elapsed = perf_counter() - start
26     end_text = (
27         f", 干燥结束={result.event_time / 3600:.6f} h"
28         if result.event_time is not None
29         else ""
30     )
31     print(
32         f"问题{problem}完成: {elapsed:.1f}s, nfev={result.nfev}{end_text}, "
33         f"末时刻中心C={result.moisture[-1, 0]:.8f}"
34     )
35
36
37  def main() -> None:
38     """问题3、4给足四天上限，事件触发后会自动停止。"""
39
40     Path(ROOT / "results").mkdir(parents=True, exist_ok=True)
41     run_one(1, 1800.0, False, 1.0, 400)
42     run_one(2, 10800.0, False, 1.0, 400)
43     run_one(3, 4.0 * 86400.0, True, 60.0, 800)
44     run_one(4, 4.0 * 86400.0, True, 60.0, 800)
45
46
47  if __name__ == "__main__":
48     main()

```

附录 C：长时结果分钟采样程序（extend_long_results.py）

```

1  """把问题3、4的结果延长到事件后的首个整60秒时刻。"""
2

```

```

3 import math
4
5 import numpy as np
6
7 from model_core import ROOT, save_result, solve_model
8
9
10 def extend(problem: int) -> None:
11     """保留连续事件时刻，同时生成严格60秒间隔的完整输出。"""
12
13     old = np.load(ROOT / f"results/q{problem}_base.npz")
14     event_time = float(old["event_time"])
15     output_end = math.ceil(event_time / 60.0) * 60.0
16     result = solve_model(
17         problem,
18         output_end,
19         n_intervals=800,
20         grid_power=2.0,
21         stop_at_dry=False,
22         rtol=1.0e-9,
23         atol=1.0e-11,
24         max_step=180.0,
25         sample_step=60.0,
26     )
27     result.event_time = event_time
28     save_result(ROOT / f"results/q{problem}_base.npz", result, problem)
29     print(
30         f"问题{problem}延长到{output_end:.0f}s；末时刻全域最大C="
31         f"{result.moisture[-1].max():.8f}"
32     )
33
34
35 def main() -> None:
36     for problem in (3, 4):
37         extend(problem)
38
39
40 if __name__ == "__main__":
41     main()

```

附录 D：结果模板填写程序（export_results.py）

```

1 """按附件3模板生成四个正式结果工作簿。"""
2
3 from __future__ import annotations
4
5 from copy import copy
6 from pathlib import Path
7
8 import numpy as np
9 import openpyxl
10
11 from model_core import ROOT
12

```

```

13 TEMPLATE_DIR = ROOT / "附件/附件3"
14 OUTPUT_DIR = ROOT / "results"
15
16
17 def interpolate_matrix(
18     field: np.ndarray, x_grid: np.ndarray, x_query: np.ndarray
19 ) -> np.ndarray:
20     """把全部时刻的场量一次性线性插值到指定坐标。"""
21
22     right = np.searchsorted(x_grid, x_query, side="right")
23     right = np.clip(right, 1, len(x_grid) - 1)
24     left = right - 1
25     width = x_grid[right] - x_grid[left]
26     weight = (x_query - x_grid[left]) / width
27     return field[:, left] * (1.0 - weight) + field[:, right] * weight
28
29
30 def copy_cell_style(source, target) -> None:
31     """扩展模板时沿用原有单元格格式。"""
32
33     target._style = copy(source._style)
34
35
36 def write_sheet(ws, times: np.ndarray, headers: list, matrix: np.ndarray) ->
    None:
37     """覆盖模板示意区并扩展为完整结果表。"""
38
39     header_style = ws.cell(1, 2)
40     time_style = ws.cell(2, 1)
41     value_style = ws.cell(2, 2)
42
43     for column, header in enumerate(headers, start=2):
44         cell = ws.cell(1, column, header)
45         copy_cell_style(header_style, cell)
46         cell.number_format = "0.0"
47
48     for row_index, time in enumerate(times, start=2):
49         time_cell = ws.cell(row_index, 1, float(time))
50         copy_cell_style(time_style, time_cell)
51         time_cell.number_format = "0"
52         for column_index, value in enumerate(matrix[row_index - 2], start=2):
53             value_cell = ws.cell(row_index, column_index, float(value))
54             copy_cell_style(value_style, value_cell)
55             value_cell.number_format = "0.0000"
56
57     ws.freeze_panes = "B2"
58     ws.sheet_view.showGridLines = False
59     ws.column_dimensions["A"].width = max(ws.column_dimensions["A"].width or 0,
60     25)
61     for column in range(2, len(headers) + 2):
62         ws.column_dimensions[openpyxl.utils.get_column_letter(column)].width =
63         11
64
65 def clear_metadata(workbook) -> None:

```

```

65     """清空输出工作簿中可能暴露身份的属性。"""
66
67     workbook.properties.creator = ""
68     workbook.properties.lastModifiedBy = ""
69     workbook.properties.title = ""
70     workbook.properties.subject = ""
71     workbook.properties.description = ""
72     workbook.properties.keywords = ""
73
74
75 def export_fixed(problem: int) -> None:
76     """导出问题1—3的固定半径结果。"""
77
78     data = np.load(OUTPUT_DIR / f"q{problem}_base.npz")
79     time_mask = data["t"] > 0.0
80     times = data["t"][time_mask]
81     positions_cm = np.round(np.arange(0.0, 2.0 + 0.05, 0.1), 1)
82     x_query = positions_cm / 2.0
83     moisture = interpolate_matrix(data["moisture"][time_mask], data["x"],
84                                  x_query)
85
86     workbook = openpyxl.load_workbook(TEMPLATE_DIR / f"result{problem}.xlsx")
87     clear_metadata(workbook)
88     if problem in (1, 2):
89         temperature = interpolate_matrix(
90             data["temperature"][time_mask], data["x"], x_query
91         )
92         write_sheet(workbook["温度"], times, positions_cm.tolist(), temperature)
93         write_sheet(workbook["水分浓度"], times, positions_cm.tolist(), moisture)
94     else:
95         write_sheet(workbook.active, times, positions_cm.tolist(), moisture)
96     workbook.save(OUTPUT_DIR / f"result{problem}.xlsx")
97
98 def export_moving() -> None:
99     """导出问题4的移动边界结果，固定位置始终位于药材内部。"""
100
101     data = np.load(OUTPUT_DIR / "q4_base.npz")
102     time_mask = data["t"] > 0.0
103     times = data["t"][time_mask]
104     radius = data["radius"][time_mask]
105     minimum_radius_cm = float(radius.min()) * 100.0
106     last_fixed = np.floor((minimum_radius_cm - 1.0e-8) * 10.0) / 10.0
107     positions_cm = np.round(np.arange(0.0, last_fixed + 0.05, 0.1), 1)
108
109     rows = []
110     field = data["moisture"][time_mask]
111     for profile, radius_m in zip(field, radius):
112         x_query = positions_cm / (radius_m * 100.0)
113         fixed_values = np.interp(x_query, data["x"], profile)
114         rows.append(np.append(fixed_values, profile[-1]))
115     matrix = np.asarray(rows)
116
117     workbook = openpyxl.load_workbook(TEMPLATE_DIR / "result4.xlsx")

```

```

118     clear_metadata(workbook)
119     headers = positions_cm.tolist() + ["药材表面"]
120     write_sheet(workbook.active, times, headers, matrix)
121     workbook.save(OUTPUT_DIR / "result4.xlsx")
122
123
124 def main() -> None:
125     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
126     for problem in (1, 2, 3):
127         export_fixed(problem)
128         print(f"result{problem}.xlsx 已生成")
129     export_moving()
130     print("result4.xlsx 已生成")
131
132
133 if __name__ == "__main__":
134     main()

```

附录 E：结果表校验程序（verify_results.py）

```

1  """核验四个结果工作簿的结构、数值映射和匿名性。"""
2
3  from __future__ import annotations
4
5  import json
6
7  import numpy as np
8  import openpyxl
9
10 from model_core import ROOT
11
12
13 def expected_value(problem: int, sheet_name: str, time: float, header) -> float:
14     """从高精度数组复算指定结果单元格。"""
15
16     data = np.load(ROOT / f"results/q{problem}_base.npz")
17     time_index = int(np.where(np.isclose(data["t"], time))[0][0])
18     field_name = "temperature" if sheet_name == "温度" else "moisture"
19     profile = data[field_name][time_index]
20     if header == "药材表面":
21         return float(profile[-1])
22     x_query = float(header) / (float(data["radius"][time_index]) * 100.0)
23     return float(np.interp(x_query, data["x"], profile))
24
25
26 def verify_book(problem: int) -> dict:
27     """核验单个结果文件。"""
28
29     path = ROOT / f"results/result{problem}.xlsx"
30     workbook = openpyxl.load_workbook(path, data_only=False, read_only=True)
31     expected_rows = {1: 1801, 2: 10801, 3: 3450, 4: 3067}[problem]
32     expected_columns = {1: 22, 2: 22, 3: 22, 4: 14}[problem]
33     step = 1 if problem in (1, 2) else 60
34     checks = {

```

```

35     "file": path.name,
36     "sheets": workbook.sheetnames,
37     "metadata_anonymous": not any(
38         [
39             workbook.properties.creator,
40             workbook.properties.title,
41             workbook.properties.subject,
42             workbook.properties.keywords,
43         ]
44     ),
45     "sheet_checks": {},
46 }
47 for ws in workbook.worksheets:
48     row_iterator = ws.iter_rows()
49     header_cells = next(row_iterator)
50     headers = [cell.value for cell in header_cells[1:]]
51     formula_count = 0
52     error_count = 0
53     times = []
54     sample_row_numbers = {2, 2 + (ws.max_row - 1) // 2, ws.max_row}
55     sample_column_numbers = {2, 2 + len(headers) // 2, ws.max_column}
56     sampled_cells = {}
57     values_last = []
58     number_format = None
59     for row_number, row in enumerate(row_iterator, start=2):
60         times.append(float(row[0].value))
61         if row_number == 2:
62             number_format = row[1].number_format
63         if row_number == ws.max_row:
64             values_last = [float(cell.value) for cell in row[1:]]
65         for column_number, cell in enumerate(row[1:], start=2):
66             if cell.data_type == "f":
67                 formula_count += 1
68             if isinstance(cell.value, str) and cell.value.startswith("#"):
69                 error_count += 1
70             if (
71                 row_number in sample_row_numbers
72                 and column_number in sample_column_numbers
73             ):
74                 sampled_cells[(row_number, column_number)] = float(cell.
75                     value)
76
77     times = np.asarray(times, dtype=float)
78     maximum_mapping_error = 0.0
79     for row in sorted(sample_row_numbers):
80         for column in sorted(sample_column_numbers):
81             actual = sampled_cells[(row, column)]
82             target = expected_value(
83                 problem,
84                 ws.title,
85                 float(times[row - 2]),
86                 headers[column - 2],
87             )
88             maximum_mapping_error = max(maximum_mapping_error, abs(actual -
89                 target))

```

```

88
89     checks["sheet_checks"][ws.title] = {
90         "shape_ok": ws.max_row == expected_rows
91         and ws.max_column == expected_columns,
92         "time_start": float(times[0]),
93         "time_end": float(times[-1]),
94         "time_step_ok": bool(np.all(np.diff(times) == step)),
95         "headers": headers,
96         "formula_count": formula_count,
97         "error_count": error_count,
98         "maximum_mapping_error": maximum_mapping_error,
99         "last_row_min": min(values_last),
100        "last_row_max": max(values_last),
101        "number_format": number_format,
102    }
103    return checks
104
105
106 def main() -> None:
107     report = {"result{problem}": verify_book(problem) for problem in (1, 2, 3,
108         4)}
109     output = ROOT / "results/xlsx_validation.json"
110     output.write_text(
111         json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8"
112     )
113     print(json.dumps(report, ensure_ascii=False, indent=2))
114
115 if __name__ == "__main__":
116     main()

```

附录 F：解析解与数值检验程序（`validate_model.py`）

```

1  """对A题求解器执行网格、容差、守恒和解析特例验证。"""
2
3  from __future__ import annotations
4
5  import json
6  from pathlib import Path
7
8  import numpy as np
9  from scipy.integrate import trapezoid
10 from scipy.optimize import brentq
11 from scipy.special import j0, j1
12
13 from model_core import (
14     C0,
15     H_MASS,
16     R0,
17     ROOT,
18     T0_C,
19     load_environment,
20     solve_model,
21 )

```

```

22
23
24 def cylinder_roots(biot: float, count: int = 50) -> np.ndarray:
25     """扫描求解无限圆柱第三类边界的特征根。"""
26
27     def equation(value: float) -> float:
28         return value * j1(value) - biot * j0(value)
29
30     grid = np.linspace(1.0e-8, (count + 8) * np.pi, (count + 8) * 80)
31     values = equation(grid)
32     roots: list[float] = []
33     for left, right, f_left, f_right in zip(
34         grid[:-1], grid[1:], values[:-1], values[1:]
35     ):
36         if f_left * f_right < 0.0:
37             root = brentq(equation, left, right)
38             if not roots or abs(root - roots[-1]) > 1.0e-8:
39                 roots.append(root)
40             if len(roots) >= count:
41                 break
42     return np.asarray(roots)
43
44
45 def analytical_temperature(x: np.ndarray, time: float, air: float) -> np.ndarray:
46     """计算恒定环境下常物性无限圆柱的解析级数解。"""
47
48     conductivity = 0.36
49     alpha = conductivity / (820.0 * 2600.0)
50     biot = 25.0 * R0 / conductivity
51     roots = cylinder_roots(biot)
52     coeff = 2.0 * j1(roots) / (roots * (j0(roots) ** 2 + j1(roots) ** 2))
53     theta = np.sum(
54         coeff[:, None]
55         * j0(roots[:, None] * x[None, :])
56         * np.exp(-(roots**2)[:, None] * alpha * time / R0**2),
57         axis=0,
58     )
59     return air + (T0_C - air) * theta
60
61
62 def control_volume_weights(x: np.ndarray) -> np.ndarray:
63     """返回与主程序一致的无量纲环形控制体权重。"""
64
65     west = np.empty_like(x)
66     east = np.empty_like(x)
67     west[0] = 0.0
68     west[1:] = 0.5 * (x[:-1] + x[1:])
69     east[:-1] = west[1:]
70     east[-1] = 1.0
71     return 0.5 * (east**2 - west**2)
72
73
74 def conservation_residual(problem: int) -> float:
75     """检查半离散水分方程的整体通量平衡。"""

```

```

76 data = np.load(ROOT / f"results/q{problem}_base.npz")
77 time = data["t"]
78 radius = data["radius"]
79 moisture = data["moisture"]
80 weight = control_volume_weights(data["x"])
81 inventory = moisture @ weight
82 _, c_air = load_environment()
83 boundary = np.asarray([c_air(float(t)) for t in time])
84 rate = H_MASS * (boundary - moisture[:, -1]) / radius
85 predicted = trapezoid(rate, time)
86 observed = inventory[-1] - inventory[0]
87 return float(abs(observed - predicted) / max(abs(observed), 1.0e-15))
88
89
90
91 def event_convergence(problem: int) -> dict[str, float]:
92     """比较径向网格加密后的干燥结束时刻。"""
93
94     values: dict[str, float] = {}
95     for n in (100, 200, 400):
96         result = solve_model(
97             problem,
98             7.0 * 86400.0,
99             n_intervals=n,
100             grid_power=2.0,
101             stop_at_dry=True,
102             rtol=1.0e-8,
103             atol=1.0e-10,
104             max_step=300.0,
105         )
106         if result.event_time is None:
107             raise RuntimeError(f"问题{problem}在计算区间内未达到干燥阈值")
108         values[str(n)] = result.event_time / 3600.0
109     return values
110
111
112 def main() -> None:
113     """汇总全部验证并保存机器可读结果。"""
114
115     constant_env = (lambda _t: 50.0, lambda _t: C0)
116     numerical = solve_model(
117         1,
118         3600.0,
119         n_intervals=400,
120         rtol=1.0e-9,
121         atol=1.0e-11,
122         max_step=30.0,
123         sample_step=600.0,
124         environment=constant_env,
125     )
126     analytical_errors = {}
127     for index, time in enumerate(numerical.t[1:], start=1):
128         exact = analytical_temperature(numerical.x, float(time), 50.0)
129         analytical_errors[str(int(time))] = float(
130             np.max(np.abs(numerical.temperature[index] - exact))

```

```

131     )
132
133     isolated = solve_model(
134         1,
135         3600.0,
136         n_intervals=50,
137         scales={"h": 0.0, "hm": 0.0},
138         sample_step=600.0,
139     )
140     isolation_error = max(
141         float(np.max(np.abs(isolated.temperature - T0_C))),
142         float(np.max(np.abs(isolated.moisture - C0))),
143     )
144
145     convergence = {"q3": event_convergence(3), "q4": event_convergence(4)}
146     tolerance = {}
147     for problem in (3, 4):
148         loose = solve_model(
149             problem,
150             7.0 * 86400.0,
151             n_intervals=200,
152             grid_power=2.0,
153             stop_at_dry=True,
154             rtol=1.0e-6,
155             atol=1.0e-8,
156             max_step=300.0,
157         )
158         tight = solve_model(
159             problem,
160             7.0 * 86400.0,
161             n_intervals=200,
162             grid_power=2.0,
163             stop_at_dry=True,
164             rtol=1.0e-8,
165             atol=1.0e-10,
166             max_step=300.0,
167         )
168         tolerance[f"q{problem}"] = abs(loose.event_time - tight.event_time) /
169             3600.0
170
171     trends = {}
172     for problem in (1, 2, 3, 4):
173         data = np.load(ROOT / f"results/q{problem}_base.npz")
174         temp_diff = np.diff(data["temperature"], axis=1)
175         moisture_diff = np.diff(data["moisture"], axis=1)
176         trends[f"q{problem}"] = {
177             "temperature_radial_violation": float(max(0.0, -temp_diff.min())),
178             "moisture_radial_violation": float(max(0.0, moisture_diff.max())),
179             "moisture_min": float(data["moisture"].min()),
180             "moisture_max": float(data["moisture"].max()),
181         }
182
183     report = {
184         "analytic_heat_max_abs_error_C": analytical_errors,
185         "zero_transfer_max_abs_error": isolation_error,

```

```

185     "event_time_hours_by_grid": convergence,
186     "event_time_tolerance_difference_hours": tolerance,
187     "moisture_balance_relative_residual": {
188         f"q{problem}": conservation_residual(problem) for problem in (1, 2,
189             3, 4)
189     },
190     "physical_trends": trends,
191 }
192 output = ROOT / "results/validation_summary.json"
193 output.write_text(
194     json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8"
195 )
196 print(json.dumps(report, ensure_ascii=False, indent=2))
197
198
199 if __name__ == "__main__":
200     main()

```

附录 G: 参数灵敏度程序 (sensitivity_analysis.py)

```

1  """实跑关键参数扰动，量化问题3和问题4结束时间的灵敏度。"""
2
3  from __future__ import annotations
4
5  import csv
6  import json
7
8  from model_core import ROOT, solve_model
9
10 PARAMETERS = {
11     "D": "水分扩散系数",
12     "hm": "表面对流传质系数",
13     "h": "表面对流换热系数",
14     "R": "半径尺度",
15 }
16 PERTURBATIONS = (-0.015, 0.015)
17
18
19 def drying_time(problem: int, scales: dict[str, float] | None = None) -> float:
20     """用统一的中等密度网格计算结束时间，单位为h。"""
21
22     result = solve_model(
23         problem,
24         7.0 * 86400.0,
25         n_intervals=150,
26         grid_power=2.0,
27         stop_at_dry=True,
28         rtol=3.0e-8,
29         atol=3.0e-10,
30         max_step=300.0,
31         scales=scales,
32     )
33     if result.event_time is None:
34         raise RuntimeError("扰动情景在计算范围内未达到干燥阈值")

```

```

35     return result.event_time / 3600.0
36
37
38 def main() -> None:
39     records = []
40     summary = {}
41     for problem in (3, 4):
42         base = drying_time(problem)
43         summary[f"q{problem}_base_hours"] = base
44         print(f"问题{problem}基准={base:.6f} h", flush=True)
45         for key, name in PARAMETERS.items():
46             for ratio in PERTURBATIONS:
47                 value = drying_time(problem, {key: 1.0 + ratio})
48                 change = (value - base) / base * 100.0
49                 elasticity = change / (ratio * 100.0)
50                 record = {
51                     "problem": problem,
52                     "parameter": key,
53                     "parameter_cn": name,
54                     "perturbation": ratio,
55                     "drying_time_h": value,
56                     "change_percent": change,
57                     "elasticity": elasticity,
58                 }
59                 records.append(record)
60                 print(
61                     f" {name}{ratio:+.1%}: {value:.6f} h " f"({change:+.4f}%)",
62                     flush=True,
63                 )
64
65     output_dir = ROOT / "results"
66     with (output_dir / "sensitivity.csv").open(
67         "w", newline="", encoding="utf-8-sig"
68     ) as file:
69         writer = csv.DictWriter(file, fieldnames=records[0].keys())
70         writer.writeheader()
71         writer.writerows(records)
72     (output_dir / "sensitivity.json").write_text(
73         json.dumps(
74             {"summary": summary, "records": records}, ensure_ascii=False, indent
75             =2
76         ),
77         encoding="utf-8",
78     )
79
80 if __name__ == "__main__":
81     main()

```

附录 H：边界稳健性程序（robustness_analysis.py）

```

1 """比较边界插值、恒温取值和半径插值口径的稳健性。"""
2
3 from __future__ import annotations

```

```

4
5 import csv
6 import json
7
8 import numpy as np
9 from scipy.interpolate import interp1d
10
11 from model_core import ROOT, _read_numeric_xlsx, load_environment, solve_model
12
13
14 def linear_environment(final_temperature: float, final_moisture: float):
15     """构造线性插值环境边界，并指定恒温阶段取值。"""
16
17     data = _read_numeric_xlsx(ROOT / "附件/附件1.xlsx")
18     t_end = float(data[-1, 0])
19     temp = interp1d(data[:, 0], data[:, 1], kind="linear")
20     moisture = interp1d(data[:, 0], data[:, 2], kind="linear")
21
22     def t_air(t: float) -> float:
23         return float(temp(t)) if t <= t_end else final_temperature
24
25     def c_air(t: float) -> float:
26         return float(moisture(t)) if t <= t_end else final_moisture
27
28     return t_air, c_air
29
30
31 def linear_radius():
32     """构造附件2的分段线性半径函数。"""
33
34     data = _read_numeric_xlsx(ROOT / "附件/附件2.xlsx")
35     time = data[:, 0]
36     radius = data[:, 1] / 100.0
37     interpolation = interp1d(time, radius, kind="linear")
38
39     def radius_fun(t: float) -> float:
40         if t <= time[-1]:
41             return float(interpolation(t))
42         return float(radius[-1])
43
44     return radius_fun
45
46
47 def run(problem: int, environment=None, radius_function=None) -> float:
48     """用一致的中等密度网格计算稳健性情景。"""
49
50     result = solve_model(
51         problem,
52         7.0 * 86400.0,
53         n_intervals=150,
54         grid_power=2.0,
55         stop_at_dry=True,
56         rtol=3.0e-8,
57         atol=3.0e-10,
58         max_step=300.0,

```

```

59     environment=environment,
60     radius_function=radius_function,
61 )
62 if result.event_time is None:
63     raise RuntimeError("稳健性情景未达到干燥阈值")
64 return result.event_time / 3600.0
65
66
67 def main() -> None:
68     pchip_environment = load_environment()
69     scenarios = [
70         ("基准PCHIP与名义恒温值", pchip_environment, None),
71         ("线性插值与名义恒温值", linear_environment(50.0, 0.05), None),
72         ("线性插值与末次实测值", linear_environment(50.165, 0.04986), None),
73         ("线性插值与末小时均值", linear_environment(49.9989, 0.04999), None),
74     ]
75     records = []
76     for problem in (3, 4):
77         base = run(problem, pchip_environment)
78         for name, environment, radius_function in scenarios:
79             value = run(problem, environment, radius_function)
80             records.append(
81                 {
82                     "problem": problem,
83                     "scenario": name,
84                     "drying_time_h": value,
85                     "difference_min": (value - base) * 60.0,
86                 }
87             )
88             print(
89                 f"问题{problem} {name}: {value:.6f} h, "
90                 f"偏差{(value-base)*60:+.3f} min",
91                 flush=True,
92             )
93         if problem == 4:
94             value = run(problem, pchip_environment, linear_radius())
95             records.append(
96                 {
97                     "problem": problem,
98                     "scenario": "半径改用线性插值",
99                     "drying_time_h": value,
100                     "difference_min": (value - base) * 60.0,
101                 }
102             )
103             print(
104                 f"问题4 半径改用线性插值: {value:.6f} h, "
105                 f"偏差{(value-base)*60:+.3f} min",
106                 flush=True,
107             )
108
109     output_dir = ROOT / "results"
110     with (output_dir / "robustness.csv").open(
111         "w", newline="", encoding="utf-8-sig"
112     ) as file:
113         writer = csv.DictWriter(file, fieldnames=records[0].keys())

```

```

114         writer.writeheader()
115         writer.writerows(records)
116         (output_dir / "robustness.json").write_text(
117             json.dumps(records, ensure_ascii=False, indent=2), encoding="utf-8"
118         )
119
120
121 if __name__ == "__main__":
122     main()

```

附录 I：论文图表生成程序（plot_results.py）

```

1  # -*- coding: utf-8 -*-
2  """生成A题论文使用的顶刊级全套学术图表（共17组，PNG 300DPI + 矢量PDF双格式）。"""
3
4  from __future__ import annotations
5
6  import json
7  from pathlib import Path
8
9  import matplotlib.pyplot as plt
10 import numpy as np
11 import openpyxl
12 from matplotlib.colors import LinearSegmentedColormap
13
14 from model_core import ROOT, material_properties
15
16 FIGURE_DIR = ROOT / "figures"
17 FIGURE_DIR.mkdir(parents=True, exist_ok=True)
18
19 # 学术顶刊配色定义
20 C_BLUE = "#2E5C8A"      # 沉稳深蓝
21 C_ORANGE = "#E8743B"    # 活力暖橙
22 C_GREEN = "#3D8B74"     # 松林雅绿
23 C_RED = "#C0392B"       # 警示绛红
24 C_CYAN = "#4A90E2"      # 天青明蓝
25 C_PURPLE = "#7D3C98"    # 典雅暮紫
26 C_GRAY = "#6E7781"      # 辅助灰
27 C_DARK = "#24292F"      # 文本深灰
28 GRID_COLOR = "#EAECEF"  # 极浅网格线
29
30 PALETTE = [C_BLUE, C_ORANGE, C_GREEN, C_RED, C_PURPLE, C_CYAN, "#E69F00"]
31
32 # 高级云图色阶（方案 A：经典顶刊雅致风，温度采用 YlOrRd 暖红，水分采用 YlGnBu 翠
33     湖蓝）
34 CMAP_TEMP = "YlOrRd"
35 CMAP_MOIST = "YlGnBu"
36
37 plt.rcParams.update(
38     {
39         "font.family": "sans-serif",
40         "font.sans-serif": ["Microsoft YaHei", "SimSun", "SimHei", "DejaVu Sans"],

```

```

40     "axes.unicode_minus": False,
41     "axes.formatter.use_mathtext": True,
42     "mathtext.fontset": "stix",
43     "pdf.fonttype": 42,
44     "ps.fonttype": 42,
45     "font.size": 10.5,
46     "axes.labelsize": 11,
47     "axes.titlesize": 12,
48     "xtick.labelsize": 10,
49     "ytick.labelsize": 10,
50     "legend.fontsize": 9.5,
51     "axes.linewidth": 0.9,
52     "lines.linewidth": 2.0,
53     "figure.facecolor": "white",
54     "axes.facecolor": "white",
55     "savefig.facecolor": "white",
56 }
57 )
58
59
60 def save_figure(fig, name: str) -> None:
61     """按竞赛要求保存高质量位图和矢量图。"""
62     fig.savefig(Figure_DIR / f"{name}.png", dpi=300, bbox_inches="tight")
63     fig.savefig(Figure_DIR / f"{name}.pdf", bbox_inches="tight")
64     plt.close(fig)
65     print(f"[图表生成] {name} -> PNG & PDF 完成")
66
67
68 def style_axis(axis) -> None:
69     """精细微弱网格线，移除顶部与右侧边框。"""
70     axis.grid(True, linestyle="--", linewidth=0.6, color=GRID_COLOR, alpha=0.9,
71               zorder=0)
72     axis.spines["top"].set_visible(False)
73     axis.spines["right"].set_visible(False)
74     axis.spines["left"].set_color("#888888")
75     axis.spines["bottom"].set_color("#888888")
76     axis.tick_params(colors=C_DARK, width=0.8)
77
78 def load_problem(problem: int):
79     return np.load(ROOT / f"results/q{problem}_base.npz")
80
81
82 def read_attachment(name: str) -> np.ndarray:
83     ws = openpyxl.load_workbook(
84         ROOT / f"附件/{name}", data_only=True, read_only=True
85     ).active
86     return np.asarray(list(ws.values)[1:], dtype=float)
87
88
89 def field_at_position(data, field_name: str, position_cm: float) -> np.ndarray:
90     values = data[field_name]
91     output = []
92     for profile, radius in zip(values, data["radius"]):
93         x_query = position_cm / (float(radius) * 100.0)

```

```

94         output.append(np.interp(x_query, data["x"], profile))
95     return np.asarray(output)
96
97
98 def environment_figure() -> None:
99     data = read_attachment("附件1.xlsx")
100     t_h = data[:, 0] / 3600.0
101     temp = data[:, 1]
102     moist = data[:, 2]
103
104     fig, axes = plt.subplots(2, 1, figsize=(7.6, 5.8), sharex=True)
105
106     # 上子图: 温度
107     axes[0].plot(t_h, temp, color=C_ORANGE, lw=2.2, label="烘房实测温度 $T_a$",
108                  zorder=3)
109     axes[0].axhline(50.0, color=C_RED, linestyle="--", lw=1.5, label="恒温设定值
110                        50.0 °C", zorder=2)
111     axes[0].fill_between(t_h, temp, 28.0, color=C_ORANGE, alpha=0.08, zorder=1)
112     axes[0].set_ylabel("烘房温度 / °C", fontweight="bold", color=C_DARK)
113     axes[0].legend(loc="lower right", frameon=True, facecolor="white", edgecolor
114                    ="#E0E0E0", framealpha=0.9)
115     style_axis(axes[0])
116
117     # 下子图: 水分浓度
118     axes[1].plot(t_h, moist, color=C_BLUE, lw=2.2, label="烘房水分浓度 $C_a$",
119                  zorder=3)
120     axes[1].axhline(0.05, color=C_GREEN, linestyle="--", lw=1.5, label="恒湿设定
121                        值 0.050 kg/kg", zorder=2)
122     axes[1].fill_between(t_h, moist, 0.015, color=C_BLUE, alpha=0.08, zorder=1)
123     axes[1].set_xlabel("烘干时间 / h", fontweight="bold", color=C_DARK)
124     axes[1].set_ylabel("水分浓度 / (kg/kg)", fontweight="bold", color=C_DARK)
125     axes[1].legend(loc="lower right", frameon=True, facecolor="white", edgecolor
126                    ="#E0E0E0", framealpha=0.9)
127     style_axis(axes[1])
128
129     fig.suptitle("烘房环境温湿度时序演化与边界接续", fontweight="bold", fontsize
130                  =13)
131     fig.tight_layout()
132     save_figure(fig, "fig01_烘房环境边界")
133
134
135 def radius_figure() -> None:
136     data = read_attachment("附件2.xlsx")
137     t_h = data[:, 0] / 3600.0
138     radius = data[:, 1]
139
140     fig, ax = plt.subplots(figsize=(7.6, 4.5))
141
142     ax.plot(t_h, radius, color=C_BLUE, lw=2.4, label="实测药材半径 $R(t)$ 曲线",
143             zorder=3)
144     ax.scatter(
145         t_h[::10],
146         radius[::10],
147         color=C_ORANGE,
148         edgecolor="white",

```

```

141         linewidth=1.2,
142         s=38,
143         zorder=4,
144         label="实测采样点",
145     )
146     ax.axhline(1.200, color=C_RED, linestyle="--", lw=1.5, label="收缩极限 $R \\approx 1.20$ cm", zorder=2)
147
148     ax.set_xlabel("烘干时间 / h", fontweight="bold")
149     ax.set_ylabel("药材截面半径 / cm", fontweight="bold")
150     ax.set_title("药材截面半径随时间收缩的动力学历程", fontweight="bold")
151     ax.set_ylim(1.05, 2.10)
152     ax.legend(loc="upper right", frameon=True, facecolor="white", edgecolor="#E0E0E0", framealpha=0.9)
153     style_axis(ax)
154     fig.tight_layout()
155     save_figure(fig, "fig02_半径收缩曲线")
156 def q1_temp_contour() -> None:
157     """问题1预热阶段药材内部温度时空分布云图（参考图配色与等温线样式）"""
158     data = load_problem(1)
159     t_grid = data["t"]
160     dist_cm = data["x"] * 2.0
161     t_full = data["temperature"].T # 形状: (401, 1801)
162
163     fig, ax = plt.subplots(figsize=(8.8, 5.0))
164     T_mesh, R_mesh = np.meshgrid(t_grid, dist_cm)
165
166     cnt = ax.contourf(T_mesh, R_mesh, t_full, levels=30, cmap="YlOrRd")
167     cbar = fig.colorbar(cnt, ax=ax)
168     cbar.set_label("药材温度 / °C", fontweight="bold", fontsize=11)
169
170     # 叠加等温线与标签
171     lines = ax.contour(
172         T_mesh,
173         R_mesh,
174         t_full,
175         levels=[29, 30, 31, 32, 33, 34, 35, 36],
176         colors="k",
177         alpha=0.35,
178         linewidths=1.0,
179     )
180     positions = [
181         (600, 1.25),
182         (850, 1.05),
183         (1050, 1.05),
184         (1250, 1.05),
185         (1420, 1.15),
186         (1560, 1.35),
187         (1650, 1.65),
188         (1740, 1.80),
189     ]
190     ax.clabel(lines, inline=True, fontsize=9, fmt="%.0f°C", manual=positions)
191
192     ax.set_xlabel("时间 / s", fontweight="bold", fontsize=12)
193     ax.set_ylabel("到药材中心距离 / cm", fontweight="bold", fontsize=12)

```

```

194     ax.set_title("图3 问题1 预热平衡阶段药材内部温度时空分布云图", fontweight="
        bold", fontsize=13)
195     fig.tight_layout()
196     save_figure(fig, "fig03_问题1温度云图")
197
198
199 def q1_moisture_profiles() -> None:
200     """问题1预热平衡阶段不同径向层位水分浓度演变历程（5条层位折线）"""
201     data = load_problem(1)
202     t_grid = data["t"]
203     c_full = data["moisture"].T # 形状: (401, 1801)
204     dist_cm = data["x"] * 2.0
205
206     fig, ax = plt.subplots(figsize=(8.5, 4.8))
207     target_dists = [0.0, 0.5, 1.0, 1.5, 2.0]
208     colors = ["#2E5C8A", "#56B4E9", "#70AD47", "#E8743B", "#D9534F"]
209
210     for d, c in zip(target_dists, colors):
211         idx = int(np.argmin(abs(dist_cm - d)))
212         lbl = (
213             r"中心  $r=0\ \mathrm{cm}$ "
214             if d == 0.0
215             else (
216                 r"表面  $r=2.0\ \mathrm{cm}$ "
217                 if d == 2.0
218                 else f" $r={d}\ \mathrm{cm}$ "
219             )
220         )
221         ax.plot(t_grid, c_full[idx, :], color=c, lw=2.2, label=lbl)
222
223     ax.set_xlabel("时间 / s", fontweight="bold", fontsize=12)
224     ax.set_ylabel("水分浓度 / (kg/kg)", fontweight="bold", fontsize=12)
225     ax.set_title("图4 问题1 预热平衡阶段不同径向层位水分浓度演变历程",
        fontweight="bold", fontsize=13)
226     ax.grid(True, linestyle="--", color="#E5E5E5", alpha=0.7)
227     ax.legend(loc="lower left", frameon=True, facecolor="white", fontsize=10.5)
228     fig.tight_layout()
229     save_figure(fig, "fig04_问题1水分云图")
230
231
232 def heatmap(problem: int, field_name: str, name: str, title: str, unit: str) ->
    None:
233     data = load_problem(problem)
234     stride = 2 if problem == 1 else 15
235     time_h = data["t"][::stride] / 3600.0
236     radius_cm = data["x"] * 2.0
237     field = data[field_name][::stride]
238
239     is_temp = "temp" in field_name.lower()
240     cmap = CMAP_TEMP if is_temp else CMAP_MOIST
241
242     fig, ax = plt.subplots(figsize=(7.6, 4.8))
243
244     mesh = ax.pcolormesh(
245         time_h, radius_cm, field.T, shading="gouraud", cmap=cmap, rasterized=

```

```

        True, zorder=1
246     )
247
248     bar = fig.colorbar(mesh, ax=ax, pad=0.025, fraction=0.045)
249     bar.set_label(unit, fontweight="bold", fontsize=10.5)
250     bar.outline.set_edgecolor("#CCCCCC")
251     bar.outline.set_linewidth(0.8)
252
253     ax.set_xlabel("烘干时间 / h", fontweight="bold")
254     ax.set_ylabel("距药材中心位置 / cm", fontweight="bold")
255     ax.set_title(title, fontweight="bold")
256     style_axis(ax)
257     fig.tight_layout()
258     save_figure(fig, name)
259
260
261 def profile_pair(problem: int, times: list[float], name: str, title: str) ->
    None:
262     data = load_problem(problem)
263     fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.4))
264     radius_cm = data["x"] * 2.0
265     use_hours = max(times) >= 3600.0
266
267     colors = [
268         "#1F77B4", "#2CA02C", "#FF7F0E", "#D62728", "#9467BD", "#8C564B", "#
            E377C2"
269     ][: len(times)]
270
271     for index, time in enumerate(times):
272         row = int(np.argmax(abs(data["t"] - time)))
273         label = f"{time / 3600:.2g} h" if use_hours else f"{time:.0f} s"
274         c = colors[index % len(colors)]
275         axes[0].plot(radius_cm, data["temperature"][row], color=c, lw=1.9, label=
            label)
276         axes[1].plot(radius_cm, data["moisture"][row], color=c, lw=1.9, label=
            label)
277
278     axes[0].set_xlabel("距中心物理距离 / cm", fontweight="bold")
279     axes[0].set_ylabel("温度 / °C", fontweight="bold")
280     axes[0].set_title("(a) 径向温度剖面演化", fontsize=11, fontweight="bold")
281
282     axes[1].set_xlabel("距中心物理距离 / cm", fontweight="bold")
283     axes[1].set_ylabel("水分浓度 / (kg/kg)", fontweight="bold")
284     axes[1].set_title("(b) 径向水分浓度剖面演化", fontsize=11, fontweight="bold"
        )
285
286     for ax in axes:
287         style_axis(ax)
288
289     # 提取统一图例放置在两子图上方的独立居中区域，彻底避免遮挡数据曲线
290     handles, labels = axes[0].get_legend_handles_labels()
291     fig.legend(
292         handles,
293         labels,
294         loc="upper center",

```

```

295         bbox_to_anchor=(0.5, 0.92),
296         ncol=len(times),
297         fontsize=9.2,
298         frameon=True,
299         facecolor="white",
300         edgecolor="#D0D0D0",
301     )
302
303     fig.suptitle(title, fontweight="bold", fontsize=12.5, y=0.98)
304     fig.tight_layout(rect=[0, 0, 1, 0.88])
305     save_figure(fig, name)
306
307
308 def q2_moisture_surface_3d() -> None:
309     """问题2 3小时烘干期水分浓度三维时空演化曲面（高保真三维立体动力学视角）"""
310     data = load_problem(2)
311     t_grid = data["t"] / 3600.0 # 转化为小时
312     dist_cm = data["x"] * 2.0 # 径向物理距离 cm
313     c_full = data["moisture"].T # 形状: (401, 10801)
314
315     # 抽样加速网格渲染并保持极高平滑度
316     t_sub = t_grid[::60]
317     c_sub = c_full[:, ::60]
318     T_mesh, R_mesh = np.meshgrid(t_sub, dist_cm)
319
320     fig = plt.figure(figsize=(9.2, 5.5))
321     ax = fig.add_subplot(111, projection="3d")
322
323     surf = ax.plot_surface(
324         T_mesh, R_mesh, c_sub, cmap="viridis", edgecolor="none", alpha=0.92
325     )
326     cbar = fig.colorbar(surf, ax=ax, shrink=0.6, aspect=12)
327     cbar.set_label("水分浓度 / (kg/kg)", fontsize=10.5, fontweight="bold")
328
329     ax.set_xlabel("时间 / h", labelpad=9, fontsize=11, fontweight="bold")
330     ax.set_ylabel("径向距离 / cm", labelpad=9, fontsize=11, fontweight="bold")
331     ax.set_zlabel("水分浓度 / (kg/kg)", labelpad=9, fontsize=11, fontweight="bold")
332     ax.set_title("问题2 3小时烘干期水分浓度三维时空演化曲面", fontsize=12.5,
333                  fontweight="bold")
334     ax.view_init(elev=28, azim=-120)
335     fig.tight_layout()
336     save_figure(fig, "fig08b_问题2三维水分曲面")
337
338 def q3_profiles() -> None:
339     data = load_problem(3)
340     times = [6, 12, 24, 36, 48, float(data["event_time"]) / 3600.0]
341     fig, ax = plt.subplots(figsize=(7.6, 4.6))
342
343     colors = [C_BLUE, "#1F968B", "#73D055", "#FDE725", C_ORANGE, C_RED]
344     for index, hour in enumerate(times):
345         row = int(np.argmax(abs(data["t"] - hour * 3600.0)))
346         is_last = index == len(times) - 1
347         label = f"达标终止时刻 ({t}* = {hour:.2f}$ h)" if is_last else f"{hour:g

```

```

        } h"
348     ax.plot(
349         data["x"] * 2.0,
350         data["moisture"][row],
351         color=colors[index % len(colors)],
352         lw=2.5 if is_last else 1.8,
353         linestyle="--" if not is_last else "-.",
354         label=label,
355         zorder=4 if is_last else 2,
356     )
357
358     ax.axhline(0.15, color=C_RED, linestyle="--", lw=1.8, label="全域干燥阈值
        0.15 kg/kg", zorder=3)
359
360     ax.set_xlabel("距中心径向距离 / cm", fontweight="bold")
361     ax.set_ylabel("水分浓度 / (kg/kg)", fontweight="bold")
362     ax.set_title("问题3（固定截面）各时刻径向水分剖面衰减演变", fontweight="bold")
363
364     ax.legend(ncol=2, fontsize=8.5, frameon=True, facecolor="white", edgecolor="
        #E0E0E0", framealpha=0.9)
365     style_axis(ax)
366     fig.tight_layout()
367     save_figure(fig, "fig09_问题3水分剖面")
368
369 def time_history(problem: int, name: str, title: str) -> None:
370     data = load_problem(problem)
371     fig, ax = plt.subplots(figsize=(7.6, 4.6))
372     positions = [0.0, 0.5, 1.0]
373
374     pos_colors = [C_BLUE, C_GREEN, C_PURPLE]
375     for index, position in enumerate(positions):
376         values = field_at_position(data, "moisture", position)
377         ax.plot(
378             data["t"] / 3600.0,
379             values,
380             color=pos_colors[index],
381             lw=2.0,
382             label=f"径向位置 $r = {position:g}$ cm",
383         )
384     ax.plot(
385         data["t"] / 3600.0,
386         data["moisture"][:, -1],
387         color=C_ORANGE,
388         lw=2.0,
389         linestyle="--",
390         label="药材外表面 ($r = R$)",
391     )
392     ax.axhline(0.15, color=C_RED, linestyle=":", lw=1.8, label="干燥阈值 0.15 kg
        /kg")
393
394     ax.set_xlabel("烘干时间 / h", fontweight="bold")
395     ax.set_ylabel("水分浓度 / (kg/kg) [对数刻度]", fontweight="bold")
396     ax.set_title(title, fontweight="bold")
397     ax.set_yscale("log")

```

```

398     ax.legend(ncol=2, fontsize=8.5, loc="upper right", frameon=True, facecolor="
        white", edgecolor="#E0E0E0", framealpha=0.9)
399     style_axis(ax)
400     fig.tight_layout()
401     save_figure(fig, name)
402
403
404 def q4_profiles() -> None:
405     data = load_problem(4)
406     times = [6, 12, 24, 36, 48, float(data["event_time"]) / 3600.0]
407     fig, ax = plt.subplots(figsize=(7.6, 4.6))
408
409     colors = [C_BLUE, "#1F968B", "#73D055", "#FDE725", C_ORANGE, C_RED]
410     for index, hour in enumerate(times):
411         row = int(np.argmin(abs(data["t"] - hour * 3600.0)))
412         radius_cm = data["x"] * data["radius"][row] * 100.0
413         is_last = index == len(times) - 1
414         label = f"达标终止时刻 ( $t^* = \{hour:.2f\}$  h)" if is_last else f"{hour:g
            } h"
415         ax.plot(
416             radius_cm,
417             data["moisture"][row],
418             color=colors[index % len(colors)],
419             lw=2.5 if is_last else 1.8,
420             linestyle="--" if not is_last else "-.",
421             label=label,
422             zorder=4 if is_last else 2,
423         )
424
425     ax.axhline(0.15, color=C_RED, linestyle="--", lw=1.8, label="全域干燥阈值
        0.15 kg/kg", zorder=3)
426     ax.set_xlabel("距药材中心的实际物理距离 / cm", fontweight="bold")
427     ax.set_ylabel("水分浓度 / (kg/kg)", fontweight="bold")
428     ax.set_title("问题4（收缩动边界）真实物理尺度下的水分剖面演化", fontweight="
        bold")
429     ax.legend(ncol=2, fontsize=8.5, frameon=True, facecolor="white", edgecolor="
        #E0E0E0", framealpha=0.9)
430     style_axis(ax)
431     fig.tight_layout()
432     save_figure(fig, "fig11_问题4水分剖面")
433
434
435 def model_comparison() -> None:
436     q3 = load_problem(3)
437     q4 = load_problem(4)
438     fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.4))
439
440     # 左图：中心水分衰减对比
441     axes[0].plot(q3["t"] / 3600.0, q3["moisture"][:, 0], color=C_BLUE, lw=2.2,
        label="问题3（固定半径 2 cm）")
442     axes[0].plot(q4["t"] / 3600.0, q4["moisture"][:, 0], color=C_ORANGE, lw=2.2,
        label="问题4（实测动态收缩半径）")
443     axes[0].axhline(0.15, color=C_RED, linestyle="--", lw=1.5, label="干燥目标阈
        值 0.15")
444     axes[0].set_xlabel("烘干时间 / h", fontweight="bold")

```

```

445 axes[0].set_ylabel("中心水分浓度 / (kg/kg)", fontweight="bold")
446 axes[0].set_title("(a) 中心水分浓度衰减历程对比", fontsize=11, fontweight="
    bold")
447 axes[0].legend(frameon=True, facecolor="white", edgecolor="#E0E0E0",
    framealpha=0.9)
448 style_axis(axes[0])
449
450 # 右图：达标时长对比柱状图
451 t3 = float(q3["event_time"]) / 3600.0
452 t4 = float(q4["event_time"]) / 3600.0
453 bars = axes[1].bar(
454     ["问题3 (固定半径)", "问题4 (动态收缩)"],
455     [t3, t4],
456     color=[C_BLUE, C_ORANGE],
457     width=0.48,
458     edgecolor="white",
459     linewidth=1.2,
460     zorder=3,
461 )
462 axes[1].set_ylabel("烘干达标时刻 / h", fontweight="bold")
463 axes[1].set_ylim(0, 68)
464 axes[1].set_title("(b) 烘干所需总时长对比", fontsize=11, fontweight="bold")
465
466 for b, val in zip(bars, [t3, t4]):
467     axes[1].text(
468         b.get_x() + b.get_width() / 2,
469         val + 1.2,
470         f"{val:.2f} h",
471         ha="center",
472         fontweight="bold",
473         fontsize=10,
474         color=C_DARK,
475     )
476 style_axis(axes[1])
477
478 fig.suptitle("几何收缩对药材热风烘干动力学进程的宏观影响", fontweight="bold",
    , fontsize=12.5)
479 fig.tight_layout()
480 save_figure(fig, "fig13_固定与收缩模型比较")
481
482
483 def convergence_figure() -> None:
484     report = json.loads((ROOT / "results/validation_summary.json").read_text("
        utf-8"))
485     fig, ax = plt.subplots(figsize=(7.6, 4.5))
486     official = {"q3": 57.472601409555914, "q4": 51.087244916844185}
487
488     markers = ["o", "s"]
489     colors = [C_BLUE, C_ORANGE]
490     for index, key in enumerate(["q3", "q4"]):
491         values = report["event_time_hours_by_grid"][key]
492         nodes = np.asarray([int(item) for item in values] + [800])
493         hours = np.asarray([values[item] for item in values] + [official[key]])
494         lbl = "问题3 (固定半径)" if key == "q3" else "问题4 (收缩半径)"
495         ax.plot(

```

```

496         nodes,
497         hours,
498         marker=markers[index],
499         markersize=6.5,
500         markerfacecolor="white",
501         markeredgewidth=1.8,
502         markeredgecolor=colors[index],
503         color=colors[index],
504         lw=2.0,
505         label=lbl,
506         zorder=3,
507     )
508
509     ax.set_xlabel("径向有限体积加密区间数 $N$", fontweight="bold")
510     ax.set_ylabel("达标烘干时长 / h", fontweight="bold")
511     ax.set_title("有限体积网格加密下的达标时刻单调收敛性（二阶表观阶）",
512                 fontweight="bold")
513     ax.legend(frameon=True, facecolor="white", edgecolor="#E0E0E0", framealpha
514              =0.9)
515     style_axis(ax)
516     fig.tight_layout()
517     save_figure(fig, "fig14_网格收敛")
518
519 def sensitivity_figure() -> None:
520     data = json.loads((ROOT / "results/sensitivity.json").read_text("utf-8"))["
521         records"]
522     labels = ["水分扩散系数 $D$", "对流传质系数 $h_m$", "对流换热系数 $h$", "药
523         材半径 $R$"]
524     param_keys = ["水分扩散系数", "表面对流传质系数", "表面对流换热系数", "半径
525         尺度"]
526
527     fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.5), sharey=True)
528
529     for axis, problem, p_title in zip(axes, (3, 4), ("(a) 问题3 敏感性响应", "(b
530         ) 问题4 敏感性响应")):
531         rows = [item for item in data if item["problem"] == problem]
532         negative = [
533             next(
534                 item["change_percent"]
535                 for item in rows
536                 if item["parameter_cn"] == key and item["perturbation"] < 0
537             )
538             for key in param_keys
539         ]
540         positive = [
541             next(
542                 item["change_percent"]
543                 for item in rows
544                 if item["parameter_cn"] == key and item["perturbation"] > 0
545             )
546             for key in param_keys
547         ]
548         y = np.arange(len(labels))
549         b1 = axis.barh(y - 0.17, negative, height=0.32, color=C_BLUE, edgecolor=

```

```

        "white", label="参数 -1.5%", zorder=3)
545     b2 = axis.barh(y + 0.17, positive, height=0.32, color=C_ORANGE,
        edgecolor="white", label="参数 +1.5%", zorder=3)
546     axis.axvline(0.0, color="#333333", linewidth=1.0, zorder=4)
547
548     axis.set_yticks(y, labels)
549     axis.set_xlabel("达标烘干时长相对变化 / %", fontweight="bold")
550     axis.set_title(p_title, fontsize=11, fontweight="bold")
551     axis.set_xlim(-3.6, 3.6)
552     style_axis(axis)
553
554     axes[1].legend(loc="lower right", fontsize=9, frameon=True, facecolor="white",
        edgecolor="#E0E0E0", framealpha=0.9)
555     fig.suptitle("关键控制参数 ±1.5% 扰动对烘干时长的影响灵敏度", fontweight="
        bold", fontsize=12.5)
556     fig.tight_layout()
557     save_figure(fig, "fig15_参数灵敏度")
558
559
560 def robustness_figure() -> None:
561     data = json.loads((ROOT / "results/robustness.json").read_text("utf-8"))
562     scenarios = [
563         "线性插值与名义恒温值",
564         "线性插值与末次实测值",
565         "线性插值与末小时均值",
566     ]
567     fig, ax = plt.subplots(figsize=(7.8, 4.6))
568     x = np.arange(len(scenarios))
569     width = 0.32
570
571     for offset, problem, color, lbl in [
572         (-width / 2, 3, C_BLUE, "问题3 时长偏差"),
573         (width / 2, 4, C_ORANGE, "问题4 时长偏差"),
574     ]:
575         rows = [item for item in data if item["problem"] == problem]
576         values = [
577             next(item["difference_min"] for item in rows if item["scenario"] ==
                    name)
578             for name in scenarios
579         ]
580         bars = ax.bar(x + offset, values, width=width, color=color, edgecolor="
            white", label=lbl, zorder=3)
581
582     ax.axhline(0.0, color="#333333", linewidth=1.0, zorder=4)
583     ax.set_xticks(
584         x, ["线性插值\n名义恒温接续", "末次实测噪声值\n永久外推接续", "末一小时
            均值\n平滑外推接续"]
585     )
586     ax.set_ylabel("相对基准达标时间的偏差 / min", fontweight="bold")
587     ax.set_title("不同边界插值与接续策略下的模型求解稳健性检验", fontweight="
        bold")
588     ax.set_ylim(-21, 5)
589     ax.legend(loc="lower left", frameon=True, facecolor="white", edgecolor="#
        E0E0E0", framealpha=0.9)
590     style_axis(ax)

```

```

591 fig.tight_layout()
592 save_figure(fig, "fig16_边界口径稳健性")
593
594
595 def diffusivity_figure() -> None:
596     concentration = np.linspace(0.05, 2.55, 500)
597     temperature = np.full_like(concentration, 50.0)
598     fig, ax = plt.subplots(figsize=(7.6, 4.6))
599
600     styles = [
601         (1, "问题1经验式  $D(C)$ ", C_BLUE, "-"),
602         (3, "问题2—3耦合式  $D(C, 50^\circ\mathrm{C})$ ", C_ORANGE, "-"),
603         (4, "问题4经验式  $D(C, 50^\circ\mathrm{C})$ ", C_GREEN, "--"),
604     ]
605
606     for problem, label, color, ls in styles:
607         diffusivity = material_properties(problem, concentration, temperature)
608         [3]
609         ax.semilogy(concentration, diffusivity, color=color, lw=2.2, linestyle=
610             ls, label=label, zorder=3)
611
612         ax.axvline(0.15, color=C_RED, linestyle="--", lw=1.8, label="干燥终止阈值
613             0.15 kg/kg", zorder=4)
614
615         ax.set_xlabel("药材干基水分浓度  $C$  / (kg/kg)", fontweight="bold")
616         ax.set_ylabel(r"水分扩散系数  $D$  / (m2/s) [对数刻度]", fontweight="bold")
617         ax.set_title("各问题水分扩散系数随浓度非线性衰减的机理特征", fontweight="
618             bold")
619         ax.legend(loc="lower right", frameon=True, facecolor="white", edgecolor="#
620             E0E0E0", framealpha=0.9)
621         style_axis(ax)
622         fig.tight_layout()
623         save_figure(fig, "fig17_扩散系数机理")
624
625
626 def main() -> None:
627     print(">>> 开始批量渲染顶刊级学术图表...")
628     environment_figure()
629     radius_figure()
630     q1_temp_contour()
631     q1_moisture_profiles()
632     profile_pair(
633         1,
634         [100, 300, 600, 900, 1200, 1500, 1800],
635         "fig05_问题1径向剖面",
636         "问题1预热阶段各采样时刻的径向温湿状态剖面",
637     )
638     heatmap(2, "temperature", "fig06_问题2温度云图", "问题2变物性耦合下内部温度
639         场时空演化", "温度 / °C")
640     heatmap(
641         2,
642         "moisture",
643         "fig07_问题2水分云图",
644         "问题2变物性耦合下内部水分浓度场时空演化",
645         "水分浓度 / (kg/kg)",

```

```

640 )
641 profile_pair(
642     2,
643     [1800, 3600, 5400, 7200, 9000, 10800],
644     "fig08_问题2径向剖面",
645     "问题2三小时内热湿耦合径向剖面动态演变",
646 )
647 q2_moisture_surface_3d()
648 q3_profiles()
649 time_history(3, "fig10_问题3水分历程", "问题3固定截面模型各径向测点水分浓度
    衰减历程")
650 q4_profiles()
651 time_history(4, "fig12_问题4水分历程", "问题4收缩动边界模型各径向测点水分浓
    度衰减历程")
652 model_comparison()
653 convergence_figure()
654 sensitivity_figure()
655 robustness_figure()
656 diffusivity_figure()
657 print(">>> 18 组顶刊级图表全部渲染完成，覆盖 PNG 与 PDF！")
658
659
660 if __name__ == "__main__":
661     main()

```

附录 J：论文结果抽取程序（generate_paper_data.py）

```

1  """从正式计算结果中生成论文表格与结果摘要。"""
2
3  from __future__ import annotations
4
5  import json
6  from pathlib import Path
7
8  import numpy as np
9
10 ROOT = Path(__file__).resolve().parent
11 RESULTS = ROOT / "results"
12
13
14 def load(problem: int):
15     """读取指定问题的正式高分辨率结果。"""
16
17     return np.load(RESULTS / f"q{problem}_base.npz")
18
19
20 def profile_at(data, time_s: float, positions_cm: list[float], field: str):
21     """对时间和径向位置作线性插值，提取论文代表值。"""
22
23     times = data["t"]
24     right = int(np.searchsorted(times, time_s))
25     if right == 0:
26         values = data[field][0]
27         radius = float(data["radius"][0])

```

```

28 elif right >= len(times):
29     values = data[field][-1]
30     radius = float(data["radius"][-1])
31 else:
32     left = right - 1
33     weight = (time_s - times[left]) / (times[right] - times[left])
34     values = (1.0 - weight) * data[field][left] + weight * data[field][right]
35     radius = float(
36         (1.0 - weight) * data["radius"][left] + weight * data["radius"][
37             right]
38     )
39     output = []
40     for position in positions_cm:
41         xi = min(position / (radius * 100.0), 1.0)
42         output.append(float(np.interp(xi, data["x"], values)))
43     return output
44
45 def markdown_table(headers: list[str], rows: list[list[object]]) -> str:
46     """生成紧凑的Markdown表格。"""
47
48     lines = ["| " + " | ".join(headers) + " |"]
49     lines.append("|" + "|".join(["---"] * len(headers)) + "|")
50     for row in rows:
51         lines.append("| " + " | ".join(str(value) for value in row) + " |")
52     return "\n".join(lines)
53
54
55 def fmt(values, digits=4):
56     """统一数值精度。"""
57
58     return [f"{value:.{digits}f}" for value in values]
59
60
61 def main() -> None:
62     q1, q2, q3, q4 = (load(i) for i in range(1, 5))
63     fixed_positions = [0.0, 0.5, 1.0, 1.5, 2.0]
64
65     sections = ["# 建模结果汇总", ""]
66     sections += [
67         "## 问题1: 常物性热湿耦合",
68         "",
69         markdown_table(
70             ["时间/s", "中心", "0.5 cm", "1.0 cm", "1.5 cm", "表面"],
71             [
72                 [time] + fmt(profile_at(q1, time, fixed_positions, "temperature"
73                                     ))
74                 for time in [100, 300, 600, 900, 1200, 1500, 1800]
75             ],
76         ),
77         "",
78         "上表为温度（℃）。水分浓度如下（kg/kg）:",
79         "",
80         markdown_table(

```

```

80         ["时间/s", "中心", "0.5 cm", "1.0 cm", "1.5 cm", "表面"],
81         [
82             [time] + fmt(profile_at(q1, time, fixed_positions, "moisture"))
83             for time in [100, 300, 600, 900, 1200, 1500, 1800]
84         ],
85     ),
86     "",
87     "## 问题2: 变物性热湿耦合",
88     "",
89     markdown_table(
90         ["时间/h", "指标", "中心", "0.5 cm", "1.0 cm", "1.5 cm", "表面"],
91         [
92             [f"{hour:g}", field_label]
93             + fmt(profile_at(q2, hour * 3600, fixed_positions, field))
94             for hour in [1, 2, 3]
95             for field, field_label in [
96                 ("temperature", "温度/°C"),
97                 ("moisture", "水分/(kg/kg)"),
98             ]
99         ],
100     ),
101     "",
102     "## 问题3: 达到干燥阈值",
103     "",
104     f"连续事件时刻: {float(q3['event_time']) / 3600:.6f} h; 提交表末时刻: {
105         float(q3['t'][-1]) / 3600:.6f} h。",
106     "",
107     markdown_table(
108         ["时间/h", "中心", "0.5 cm", "1.0 cm", "1.5 cm", "表面"],
109         [
110             [f"{hour:g}"]
111             + fmt(profile_at(q3, hour * 3600, fixed_positions, "moisture"))
112             for hour in [6, 12, 18, 24, 36, 48]
113         ]
114         + [
115             ["事件时刻"]
116             + fmt(
117                 profile_at(
118                     q3,
119                     float(q3["event_time"]),
120                     fixed_positions,
121                     "moisture",
122                 )
123             ),
124         ],
125     ),
126     "",
127     "## 问题4: 收缩边界",
128     "",
129     f"连续事件时刻: {float(q4['event_time']) / 3600:.6f} h; 提交表末时刻: {
130         float(q4['t'][-1]) / 3600:.6f} h; 事件半径: {float(np.interp(float(
131             q4['event_time']), q4['t'], q4['radius'])) * 100:.4f} cm。",

```

```

水分"],
132     [
133         [
134             f"{hour:g}",
135             f"{float(np.interp(hour * 3600, q4['t'], q4['radius'])) *
                100:.4f}",
136         ]
137     + fmt(profile_at(q4, hour * 3600, [0.0, 0.5, 1.0, 2.0], "
        moisture"))
138     for hour in [6, 12, 18, 24, 36, 48]
139     ]
140     + [
141         [
142             "事件时刻",
143             f"{float(np.interp(float(q4['event_time']), q4['t'], q4['
                radius'])) * 100:.4f}",
144         ]
145     + fmt(
146         profile_at(
147             q4,
148             float(q4["event_time"]),
149             [0.0, 0.5, 1.0, 2.0],
150             "moisture",
151         )
152     )
153     ],
154 ),
155 "",
156 "注：问题4中超过当时半径的位置自动映射为表面，仅用于本摘要；正式result4.
    xlsx严格按题面固定位置加动态表面输出。",
157 "",
158 ]
159
160 validation = json.loads((RESULTS / "validation_summary.json").read_text("utf
    -8"))
161 sensitivity = json.loads((RESULTS / "sensitivity.json").read_text("utf-8"))
162 robustness = json.loads((RESULTS / "robustness.json").read_text("utf-8"))
163 mechanism = json.loads(
164     (RESULTS / "mechanism_decomposition.json").read_text("utf-8")
165 )
166 sections += [
167     "## 数值验证与稳健性",
168     "",
169     "```json",
170     json.dumps(
171         {
172             "validation": validation,
173             "sensitivity": sensitivity,
174             "robustness": robustness,
175             "mechanism_decomposition": mechanism,
176         },
177         ensure_ascii=False,
178         indent=2,
179     ),
180     "```",

```

```

181     "",
182 ]
183 (ROOT / "建模结果汇总.md").write_text("\n".join(sections), encoding="utf-8")
184
185 macros = [
186     "% 本文件由generate_paper_data.py自动生成",
187     f"\\newcommand{{\\QThreeEventHours}}{{float(q3['event_time']) / 3600:.6f}}",
188     f"\\newcommand{{\\QFourEventHours}}{{float(q4['event_time']) / 3600:.6f}}",
189     f"\\newcommand{{\\QThreeOutputSeconds}}{{int(q3['t'][-1])}}",
190     f"\\newcommand{{\\QFourOutputSeconds}}{{int(q4['t'][-1])}}",
191     f"\\newcommand{{\\QFourEventRadius}}{{float(np.interp(float(q4['event_time']), q4['t'], q4['radius'])) * 100:.4f}}",
192     f"\\newcommand{{\\QTwoFinalCenterT}}{{profile_at(q2, 10800, [0.0], 'temperature')[0]:.4f}}",
193     f"\\newcommand{{\\QTwoFinalSurfaceT}}{{profile_at(q2, 10800, [2.0], 'temperature')[0]:.4f}}",
194     f"\\newcommand{{\\QTwoFinalCenterC}}{{profile_at(q2, 10800, [0.0], 'moisture')[0]:.4f}}",
195     f"\\newcommand{{\\QTwoFinalSurfaceC}}{{profile_at(q2, 10800, [2.0], 'moisture')[0]:.4f}}",
196 ]
197 (ROOT / "paper_data.tex").write_text("\n".join(macros) + "\n", encoding="utf-8")
198
199
200 if __name__ == "__main__":
201     main()

```

附录 K：支撑材料整理程序（prepare_support.py）

```

1  """整理支撑材料、生成附录清单并压缩。"""
2
3  from __future__ import annotations
4
5  import shutil
6  import zipfile
7  from pathlib import Path
8
9  import numpy as np
10
11  ROOT = Path(__file__).resolve().parent
12  STAGING = ROOT / "support_materials"
13  ZIP_PATH = ROOT.parent / "支撑材料.zip"
14
15  CODE_FILES = [
16      "model_core.py",
17      "solve_all.py",
18      "extend_long_results.py",
19      "export_results.py",
20      "verify_results.py",
21      "validate_model.py",
22      "sensitivity_analysis.py",

```

```

23     "robustness_analysis.py",
24     "decompose_effects.py",
25     "plot_results.py",
26     "generate_paper_data.py",
27     "prepare_support.py",
28 ]
29
30
31 def copy_file(source: Path, relative: Path) -> None:
32     """复制文件并保留清晰的相对目录。"""
33
34     destination = STAGING / relative
35     destination.parent.mkdir(parents=True, exist_ok=True)
36     shutil.copy2(source, destination)
37
38
39 def compact_npz(problem: int) -> None:
40     """保存论文复核用的压缩快照，完整场可由主程序重算。"""
41
42     source = np.load(ROOT / f"results/q{problem}_base.npz")
43     time_stride = max(1, int(round(60.0 / np.median(np.diff(source["t"]))))))
44     radial_stride = max(1, (len(source["x"]) - 1) // 100)
45     time_idx = np.arange(0, len(source["t"]), time_stride)
46     if time_idx[-1] != len(source["t"]) - 1:
47         time_idx = np.append(time_idx, len(source["t"]) - 1)
48     radial_idx = np.arange(0, len(source["x"]), radial_stride)
49     if radial_idx[-1] != len(source["x"]) - 1:
50         radial_idx = np.append(radial_idx, len(source["x"]) - 1)
51
52     target = STAGING / "results" / f"q{problem}_paper_snapshot.npz"
53     target.parent.mkdir(parents=True, exist_ok=True)
54     np.savez_compressed(
55         target,
56         t=source["t"][time_idx].astype(np.float32),
57         x=source["x"][radial_idx].astype(np.float32),
58         radius=source["radius"][time_idx].astype(np.float32),
59         temperature=source["temperature"][np.ix_(time_idx, radial_idx)].astype(
60             np.float32
61         ),
62         moisture=source["moisture"][np.ix_(time_idx, radial_idx)].astype(np.
63             float32),
64         event_time=np.asarray(source["event_time"], dtype=np.float64),
65     )
66
67 def tex_escape(text: str) -> str:
68     """转义附录文件名中的LaTeX特殊字符。"""
69
70     return (
71         text.replace("\\", "/")
72         .replace("_", r"\_")
73         .replace("%", r"\%")
74         .replace("#", r"\#")
75         .replace("&", r"\&")
76     )

```

```

77
78
79 def purpose(path: str) -> str:
80     """按目录给出简明用途。"""
81
82     if path.endswith("AI工具使用详情.pdf"):
83         return "AI工具使用过程、采纳与人工核验说明"
84     if path.startswith("figures/"):
85         return "论文图表的位图或矢量版本"
86     if path.startswith("results/"):
87         return "正式结果表、压缩场快照或校验记录"
88     if path.startswith("附件/"):
89         return "模型运行所需的题目输入数据"
90     if path.endswith(".py"):
91         return "模型求解、验证、出图或材料整理源程序"
92     return "建模结果与复核摘要"
93
94
95 def write_tex_list(files: list[str]) -> None:
96     """生成与压缩包逐项一致的三线表清单。"""
97
98     groups = {
99         "源程序与说明": [p for p in files if "/" not in p],
100         "输入数据": [p for p in files if p.startswith("附件/)],
101         "图表文件": [p for p in files if p.startswith("figures/)],
102         "结果与核验文件": [p for p in files if p.startswith("results/)],
103     }
104     lines = []
105     for title, paths in groups.items():
106         lines += [
107             r"\begin{longtable}{p{0.58\textwidth}p{0.34\textwidth}}",
108             rf"\caption*{{{title}}}\\",
109             r"\toprule 文件 & 用途\\",
110             r"\midrule\endfirsthead",
111             r"\toprule 文件 & 用途\\",
112             r"\midrule\endhead",
113         ]
114         for path in paths:
115             lines.append(f"{tex_escape(path)} & {purpose(path)}\\")
116         lines += [r"\bottomrule", r"\end{longtable}", ""]
117     (ROOT / "paper/support_file_list.tex").write_text(
118         "\n".join(lines), encoding="utf-8"
119     )
120
121
122 def main() -> None:
123     STAGING.mkdir(parents=True, exist_ok=True)
124     for name in CODE_FILES:
125         copy_file(ROOT / name, Path(name))
126     copy_file(ROOT / "建模结果汇总.md", Path("建模结果汇总.md"))
127     copy_file(ROOT / "paper/ai_usage.pdf", Path("AI工具使用详情.pdf"))
128
129     for name in ["附件1.xlsx", "附件2.xlsx"]:
130         copy_file(ROOT / f"附件/{name}", Path("附件") / name)
131     for source in sorted((ROOT / "figures").glob("*")):

```

```

132     if source.suffix.lower() in {".png", ".pdf"}:
133         copy_file(source, Path("figures") / source.name)
134
135     result_names = [
136         "result1.xlsx",
137         "result2.xlsx",
138         "result3.xlsx",
139         "result4.xlsx",
140         "validation_summary.json",
141         "sensitivity.csv",
142         "sensitivity.json",
143         "robustness.csv",
144         "robustness.json",
145         "mechanism_decomposition.json",
146         "xlsx_validation.json",
147     ]
148     for name in result_names:
149         copy_file(ROOT / f"results/{name}", Path("results") / name)
150     for problem in range(1, 5):
151         compact_npz(problem)
152
153     files = sorted(
154         path.relative_to(STAGING).as_posix()
155         for path in STAGING.rglob("*")
156         if path.is_file()
157     )
158     write_tex_list(files)
159
160     with zipfile.ZipFile(
161         ZIP_PATH, "w", compression=zipfile.ZIP_DEFLATED, compresslevel=9
162     ) as archive:
163         for relative in files:
164             archive.write(STAGING / relative, relative)
165     print(f"支撑材料: {len(files)}个文件, {ZIP_PATH.stat().st_size / 1024**2:.2f} MB")
166
167
168 if __name__ == "__main__":
169     main()

```

附录 L: 物性与收缩贡献分解程序 (decompose_effects.py)

```

1  """分离问题4中物性公式变化与半径收缩对达标时长的贡献。"""
2
3  from __future__ import annotations
4
5  import json
6
7  import numpy as np
8
9  from model_core import ROOT, solve_model
10
11
12 def main() -> None:

```

```

13 """ 计算问题4物性、固定2 cm半径的反事实基准。 """
14
15 q3 = np.load(ROOT / "results/q3_base.npz")
16 q4 = np.load(ROOT / "results/q4_base.npz")
17 fixed_times = {}
18 for intervals in (200, 400):
19     result = solve_model(
20         4,
21         400.0 * 3600.0,
22         n_intervals=intervals,
23         stop_at_dry=True,
24         rtol=1.0e-7,
25         atol=1.0e-9,
26         max_step=600.0,
27         grid_power=2.0,
28         radius_function=lambda _t: 0.02,
29     )
30     if result.event_time is None:
31         raise RuntimeError("反事实场景在计算区间内未达到阈值")
32     fixed_times[str(intervals)] = result.event_time / 3600.0
33
34 q3_hours = float(q3["event_time"]) / 3600.0
35 q4_hours = float(q4["event_time"]) / 3600.0
36 fixed_q4_hours = fixed_times["400"]
37 summary = {
38     "q3_properties_fixed_radius_hours": q3_hours,
39     "q4_properties_fixed_radius_hours": fixed_q4_hours,
40     "q4_properties_shrinking_radius_hours": q4_hours,
41     "property_formula_contribution_hours": fixed_q4_hours - q3_hours,
42     "shrinkage_contribution_hours": q4_hours - fixed_q4_hours,
43     "combined_difference_hours": q4_hours - q3_hours,
44     "q4_fixed_radius_grid_check_hours": fixed_times,
45 }
46 path = ROOT / "results/mechanism_decomposition.json"
47 path.write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding=
    "utf-8")
48 print(json.dumps(summary, ensure_ascii=False, indent=2))
49
50
51 if __name__ == "__main__":
52     main()

```